G. N. KHALSA COLLEGE

AUTONOMOUS

(UNIVERSITY OF MUMBAI)

MUMBAI-400019

DEPARTMENT OF COMPUTER SCIENCE

CERTIFICATE

Exam Seat No. __369__

CERTIFIED that the practicals, assignment duly signed, were performed by

Mr. ___**Balmukund Shukla**_____Roll No. _369_of M.Sc Part I/Part II class in the Computer Science Laboratory of G. N. Khalsa College, Mumbai during academic year 2026-2027.

He/She has completed the course of Laboratory assignments in Computer Science as contained in the course prescribed by the University of Mumbai

Sign. Of the Student

Date_____

Head of Dept.

Computer Science

Date_____

Professor-in-charge

1)_____

Date_____

2)_____

Date_____

Sign of Examiner's

1)_____

Date_____

2)_____

Date_____

Image Processing + Advanced Databases

INDEX

SR. NO.	TITLE	DATE	PG.NO.	SIGNATURE
	DIGITAL SIGNAL AND IMAGE PROCESSING			
1	a. Generate and visualize sine, cosine, square, sawtooth signals b. Effect of sampling frequency on sinusoidal signal c. Add Gaussian noise; compute Mean, Variance, SNR d. Compare moving-average and frequency-domain low-pass smoothing	01-08-26	1-5	
2	a. Read, display, analyze, modify, save image using OpenCV b. Intensity transformations: Negative, Logarithmic, Gamma	08-08-26	6-8	
3	a. Contrast stretching and global/adaptive thresholding b. Histogram equalization and histogram matching	22-08-26	9-12	
4	a. Remove salt-and-pepper noise using Mean, Gaussian, Median filters b. Edge/detail enhancement using Laplacian and high-pass sharpening c. To calculate image gradients using Sobel operator and Laplacian	29-08-26	13-18	

ADVANCED DATABASE TECHNIQUES				
1	Demonstrate Object-Oriented Databases	05-09-26	17-19	
2	Demonstrate a distributed database environment by dividing a given global database table into 1. Horizontal fragments 2. Vertical fragments	12-09-26	20-31	
3	Perform the CRUD operations using 1. MongoDB 2. CouchDB	19-09-26	32-44	
4	Perform the CRUD operations using 1. Redis 2. Apache Cassandra	26-09-26	45-56	

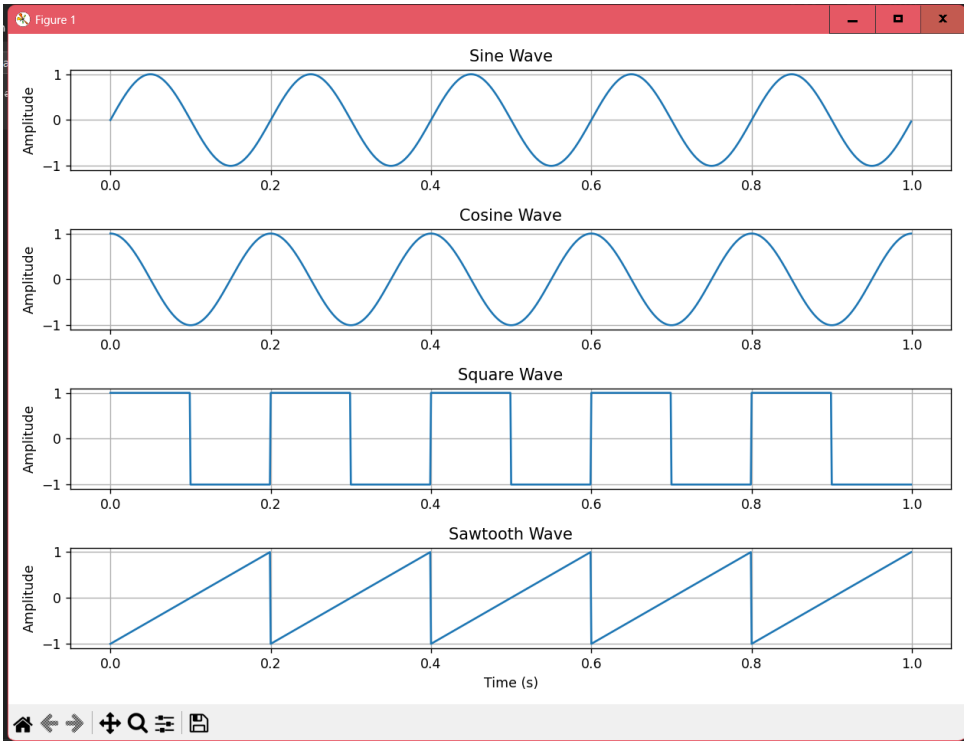
Practical No: 01

1A) Generate and visualize sine, cosine, square, sawtooth signals

Code:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal
fs = 1000          # Sampling frequency (Hz)
t = np.arange(0, 1, 1/fs)  # Time vector from 0 to 1 second
f = 5              # Analog frequency (Hz)
# Generating the waves
sine = np.sin(2 * np.pi * f * t)
cosine = np.cos(2 * np.pi * f * t)
square = signal.square(2 * np.pi * f * t)
sawtooth = signal.sawtooth(2 * np.pi * f * t)
# Plotting
plt.figure(figsize=(10, 7))
plt.subplot(4, 1, 1)
plt.plot(t, sine)
plt.title("Sine Wave")
plt.ylabel("Amplitude")
plt.grid()
plt.subplot(4, 1, 2)
plt.plot(t, cosine)
plt.title("Cosine Wave")
plt.ylabel("Amplitude")
plt.grid()
plt.subplot(4, 1, 3)
plt.plot(t, square)
plt.title("Square Wave")
plt.ylabel("Amplitude")
plt.grid()
plt.subplot(4, 1, 4)
plt.plot(t, sawtooth)
plt.title("Sawtooth Wave")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude")
plt.grid()
plt.tight_layout()
plt.show()
```

Output:



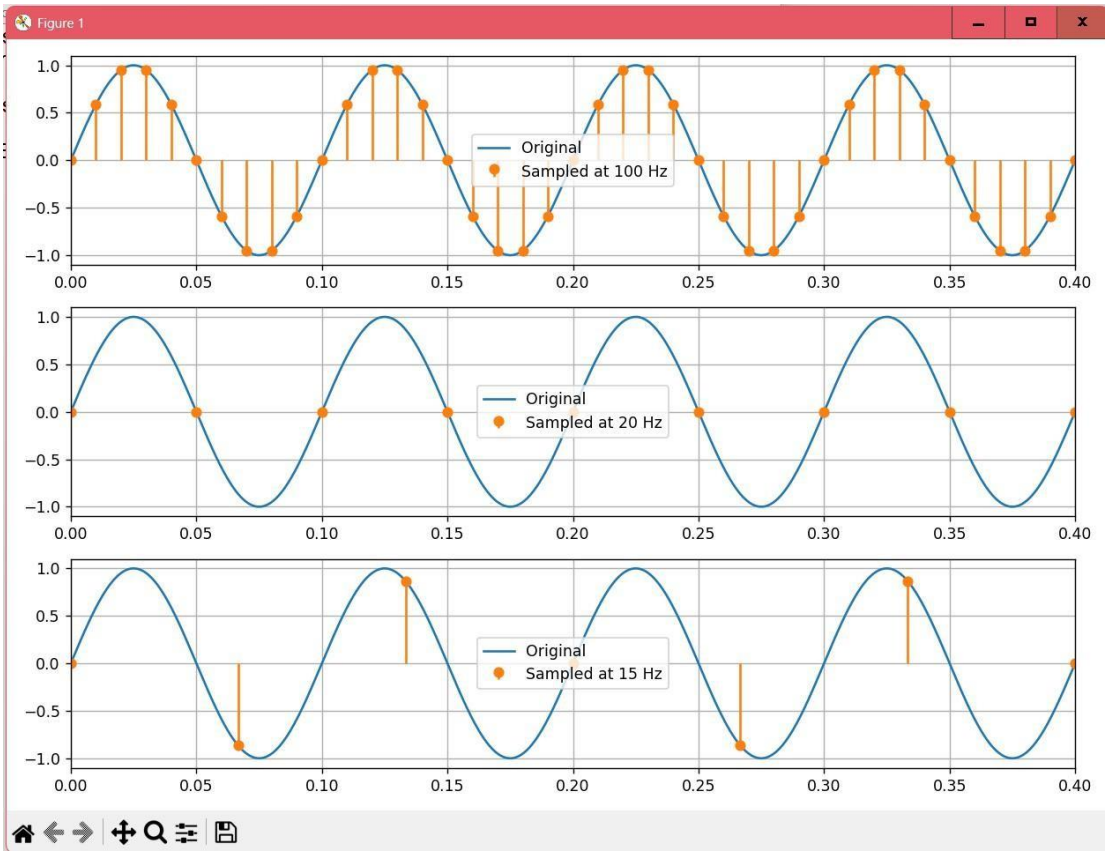
1B) Effect of sampling frequency on sinusoidal signal

Code:

```
import numpy as np
import matplotlib.pyplot as plt
f = 10  # Analog signal frequency (Hz)
t_cont = np.linspace(0, 1, 2000)  # Super high-res "continuous" time
x_cont = np.sin(2*np.pi*f*t_cont)  # The original perfect wave
sampling_rates = [100, 20, 15]  # Different sampling frequencies (Hz)
plt.figure(figsize=(10, 7))
for i, fs in enumerate(sampling_rates, 1):
    t = np.arange(0, 1, 1/fs)  # Discrete time points
    x = np.sin(2*np.pi*f*t)    # Sampled signal
    plt.subplot(3, 1, i)
```

```
    plt.plot(t_cont, x_cont, label="Original")  # Smooth line
    plt.stem(t, x, linefmt="C1-", markerfmt="C1o", basefmt=" ", label=f"Sampled at {fs} Hz")
    plt.xlim(0, 0.4)  # Zoom in to first 0.4 seconds
    plt.grid()
    plt.legend()
plt.tight_layout()
plt.show()
```

Output:

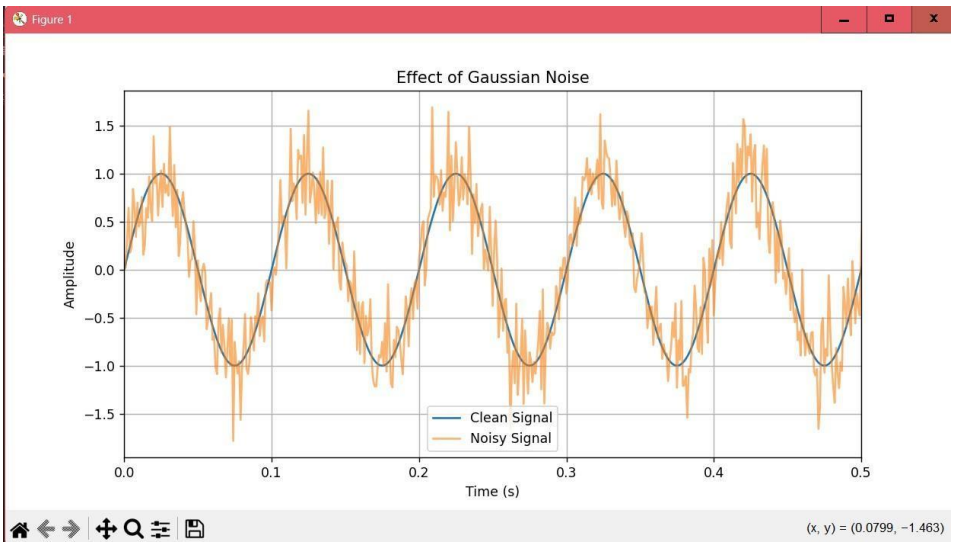


1C) Add Gaussian noise; compute Mean, Variance, SNR

Code:

```
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(42)  # Freezing the random generator so I get the same "random" results every time
fs = 1000
t = np.arange(0, 1, 1/fs)
clean = np.sin(2*np.pi*10*t)                # Pure 10 Hz signal
noise = np.random.normal(0, 0.3, len(t))    # Gaussian noise: mean=0, std=0.3
noisy = clean + noise                       # Corrupted signal
# Calculating SNR
signal_power = np.mean(clean**2)
noise_power = np.mean(noise**2)
snr = 10 * np.log10(signal_power / noise_power)
print("Mean of noise:", np.mean(noise))
print("Variance of noise:", np.var(noise))
print("SNR:", round(snr, 2), "dB")
plt.figure(figsize=(10, 5))
plt.plot(t, clean, label="Clean Signal")
plt.plot(t, noisy, alpha=0.6, label="Noisy Signal")
plt.xlim(0, 0.5)
plt.xlabel("Time (s)")
plt.ylabel("Amplitude")
plt.title("Effect of Gaussian Noise")
plt.grid()
plt.legend()
plt.show()
```

Output:



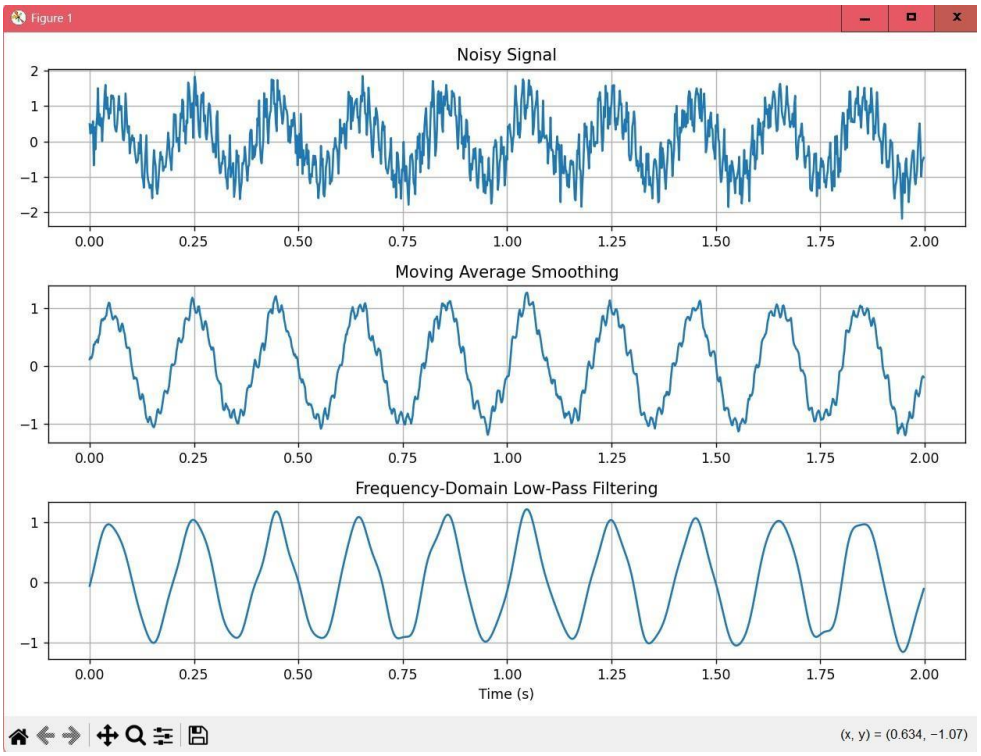

```
---------- RESTART: C:/Users/Shikha/
Mean of noise: 0.005799616746697651
Variance of noise: 0.08621144907583655
SNR: 7.63 dB
```

1D) Compare moving-average and frequency-domain low-pass smoothing

Code:

```
import numpy as np
import matplotlib.pyplot as plt
fs = 500
t = np.arange(0, 2, 1/fs)
np.random.seed(1)
# Creating a signal with a low frequency (5 Hz), a high frequency (60 Hz), and noise
x = np.sin(2*np.pi*5*t) + 0.5*np.sin(2*np.pi*60*t)
x += 0.3*np.random.randn(len(t))
kernel = np.ones(11) / 11   # Smoothing window of size 11
smooth = np.convolve(x, kernel, mode="same")
# --- Frequency-domain low-pass filter ---
X = np.fft.rfft(x)
freq = np.fft.rfftfreq(len(x), 1/fs)
cutoff = 20                 # Cut out everything above 20 Hz
X_filtered = X.copy()
X_filtered[freq > cutoff] = 0
filtered = np.fft.irfft(X_filtered, n=len(x))
# Plotting
plt.figure(figsize=(10, 7))
plt.subplot(3, 1, 1)
plt.plot(t, x)
plt.title("Noisy Signal")
plt.grid()
plt.subplot(3, 1, 2)
plt.plot(t, smooth)
plt.title("Moving Average Smoothing")
plt.grid()
plt.subplot(3, 1, 3)
plt.plot(t, filtered)
plt.title("Frequency-Domain Low-Pass Filtering")
plt.xlabel("Time (s)")
plt.grid()
plt.tight_layout()
plt.show()
```

Output:


Figure 1
Noisy Signal
Moving Average Smoothing
Frequency-Domain Low-Pass Filtering
Time (s)
(x, y) = (0.634, −1.07)

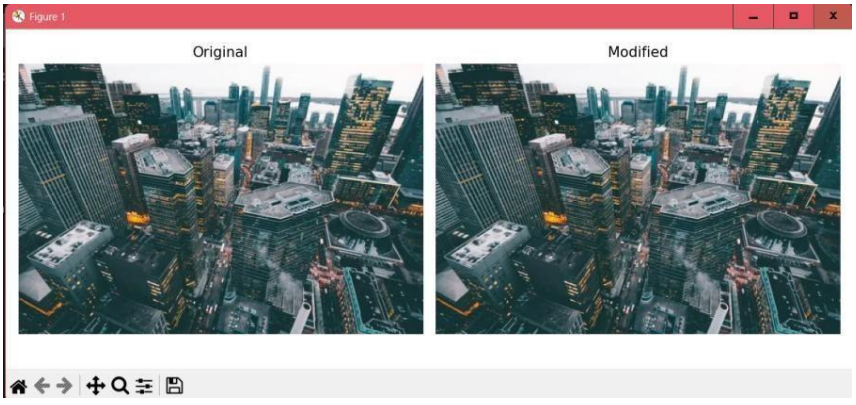
Practical No: 02

2A) Read, display, analyze, modify, save image using OpenCV

Code:

```
import cv2
import matplotlib.pyplot as plt
# Read the image
img = cv2.imread("input.jpg")
if img is None:
    raise FileNotFoundError("Place an image named input.jpg in the same folder.")
# Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
# Print image properties
print("Image shape:", img.shape)
print("Data type:", img.dtype)
print("Number of pixels:", img.shape[0] * img.shape[1])
# Modify one pixel (turn it red in BGR format)
modified = img.copy()
modified[100, 100] = [0, 0, 255]  # BGR: Blue=0, Green=0, Red=255
# Save the modified image
cv2.imwrite("output.png", modified)
# Display the images side by side
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Original")
plt.axis("off")
plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(modified, cv2.COLOR_BGR2RGB))
plt.title("Modified")
plt.axis("off")
plt.tight_layout()
plt.show()
```

Output:



2B) Intensity transformations: Negative, Logarithmic, Gamma

Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image as grayscale
gray = cv2.imread("input.jpg", cv2.IMREAD_GRAYSCALE)

if gray is None:
    raise FileNotFoundError("Place an image named input.jpg in the same folder.")

# --- 1. Negative Transformation ---
negative = 255 - gray

# --- 2. Log Transformation ---
c = 255 / np.log1p(np.max(gray))  # Scaling constant so output is in 0-255 range
log_img = c * np.log1p(gray)       # log1p is log(1+x) to avoid log(0)
log_img = np.uint8(np.clip(log_img, 0, 255))

# --- 3. Power-Law (Gamma) Transformation ---
gamma = 0.5
normalized = gray / 255.0          # Convert to range [0, 1]
gamma_img = np.uint8(255 * (normalized ** gamma))

# Plotting the results
plt.figure(figsize=(12, 7))

images = [gray, negative, log_img, gamma_img]
titles = ["Original", "Negative", "Log Transform", f"Gamma = {gamma}"]

for i, (image, title) in enumerate(zip(images, titles), 1):
    plt.subplot(2, 2, i)
    plt.imshow(image, cmap="gray")
    plt.title(title)
    plt.axis("off")

plt.tight_layout()
plt.show()
```

Output:


Figure 1
Original
Negative
Log Transform
Gamma = 0.5
(x, y) = (396.3, 201.4)
[42.0]

Practical No: 03

3A) Contrast stretching and global/adaptive thresholding

Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
gray = cv2.imread("input.jpg", cv2.IMREAD_GRAYSCALE)

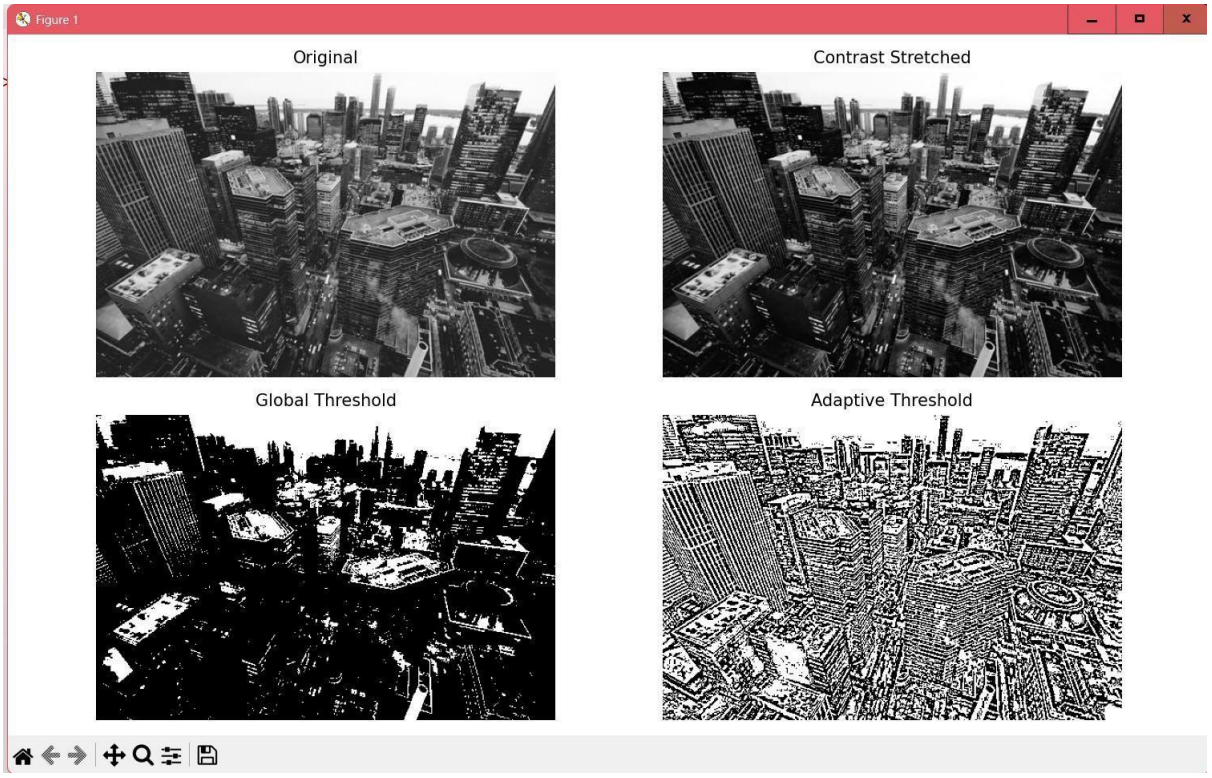
if gray is None:
    raise FileNotFoundError("Place an image named input.jpg in the same folder.")
#  Contrast Stretching
r_min, r_max = np.percentile(gray, (2, 98))  # Ignore extreme outliers
stretched = (gray - r_min) * 255 / (r_max - r_min)
stretched = np.uint8(np.clip(stretched, 0, 255))
#  Global Thresholding
_, binary = cv2.threshold(stretched, 127, 255, cv2.THRESH_BINARY)
#  Adaptive Thresholding
adaptive = cv2.adaptiveThreshold(
    stretched, 255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY, 11, 2
)
# Plotting
plt.figure(figsize=(12, 7))

images = [gray, stretched, binary, adaptive]
titles = ["Original", "Contrast Stretched", "Global Threshold", "Adaptive Threshold"]

for i, (image, title) in enumerate(zip(images, titles), 1):
    plt.subplot(2, 2, i)
    plt.imshow(image, cmap="gray")
    plt.title(title)
    plt.axis("off")

plt.tight_layout()
plt.show()
```

Output:


Figure 1
Original
Contrast Stretched
Global Threshold
Adaptive Threshold

3B) Histogram equalization and histogram matching

Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage.exposure import match_histograms
# Read source and reference images as grayscale
source = cv2.imread("input.jpg", cv2.IMREAD_GRAYSCALE)
reference = cv2.imread("reference.jpg", cv2.IMREAD_GRAYSCALE)
if source is None or reference is None:
    raise FileNotFoundError("Place input.jpg and reference.jpg in the same folder.")

#  Histogram Equalization
equalized = cv2.equalizeHist(source)

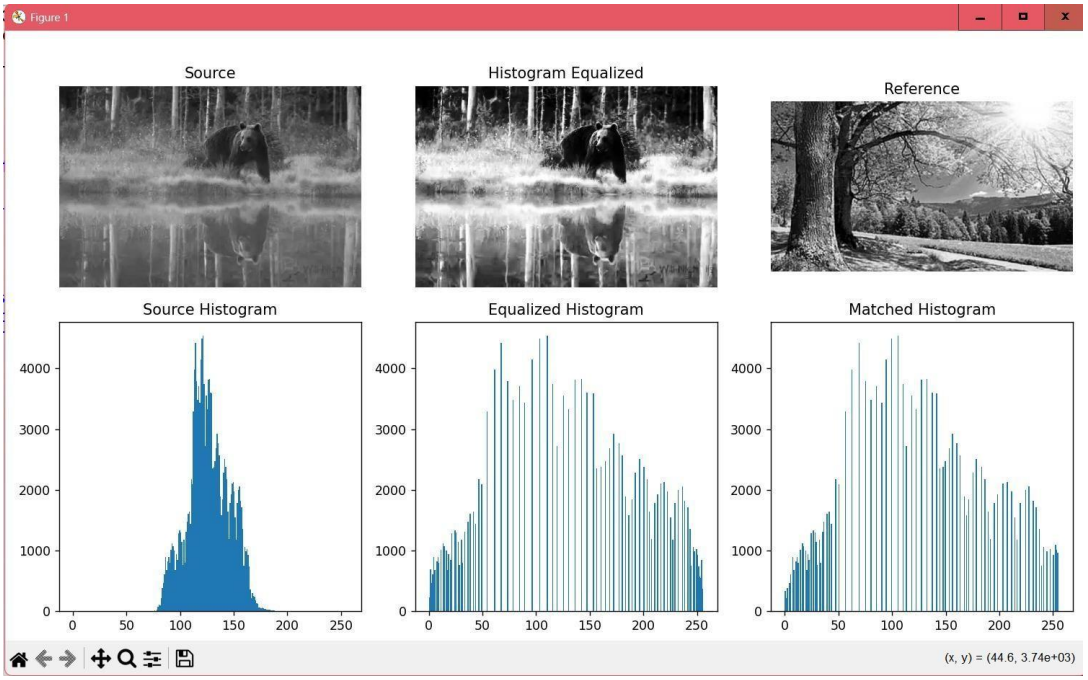
#  Histogram Matching
matched = match_histograms(source, reference)
matched = np.uint8(np.clip(matched, 0, 255))

# Plotting images and their histograms
plt.figure(figsize=(12, 8))

# Row 1: Images
plt.subplot(2, 3, 1)
plt.imshow(source, cmap="gray")
plt.title("Source")
plt.axis("off")
plt.subplot(2, 3, 2)
plt.imshow(equalized, cmap="gray")
plt.title("Histogram Equalized")
plt.axis("off")
plt.subplot(2, 3, 3)
plt.imshow(reference, cmap="gray")
plt.title("Reference")
plt.axis("off")
# Row 2: Histograms
plt.subplot(2, 3, 4)
plt.hist(source.ravel(), 256, [0, 256])
```

```
plt.title("Source Histogram")
plt.subplot(2, 3, 5)
plt.hist(equalized.ravel(), 256, [0, 256])
plt.title("Equalized Histogram")
plt.subplot(2, 3, 6)
plt.hist(matched.ravel(), 256, [0, 256])
plt.title("Matched Histogram")
plt.tight_layout()
plt.show()
```

Output:


Figure 1
Source
Histogram Equalized
Reference
Source Histogram
Equalized Histogram
Matched Histogram
4000
3000
2000
1000
0
0
50
100
150
200
250
(x, y) = (44.6, 3.74e+03)

Practical No: 04

4A) Remove salt-and-pepper noise using Mean, Gaussian, Median filters

Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("input.jpg", cv2.IMREAD_GRAYSCALE)

if img is None:
    raise FileNotFoundError("Place input.jpg in the same folder.")

# --- Adding Salt-and-Pepper Noise ---
np.random.seed(42)
noisy = img.copy()

amount = 0.05  # 5% of pixels will be corrupted
num_pixels = int(amount * img.size)

# Add "salt" (white pixels)
coords = (
    np.random.randint(0, img.shape[0], num_pixels),
    np.random.randint(0, img.shape[1], num_pixels)
)
noisy[coords] = 255

# Add "pepper" (black pixels)
coords = (
    np.random.randint(0, img.shape[0], num_pixels),
    np.random.randint(0, img.shape[1], num_pixels)
)
noisy[coords] = 0

# --- Applying Filters ---
mean_filter = cv2.blur(noisy, (5, 5))              # Linear: simple average
gaussian = cv2.GaussianBlur(noisy, (5, 5), 0)      # Linear: weighted average
median = cv2.medianBlur(noisy, 5)                  # Non-linear: median value

# --- Plotting ---
plt.figure(figsize=(12, 8))
images = [img, noisy, mean_filter, gaussian, median]
titles = ["Original", "Noisy", "Mean Filter",
          "Gaussian Filter", "Median Filter"]

for i, (image, title) in enumerate(zip(images, titles), 1):
```

```
    plt.subplot(2, 3, i)
    plt.imshow(image, cmap="gray")
    plt.title(title)
    plt.axis("off")

plt.tight_layout()
plt.show()
```

Output:



4B) Edge/detail enhancement using Laplacian and high-pass sharpening

Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

gray = cv2.imread("input.jpg", cv2.IMREAD_GRAYSCALE)

if gray is None:
    raise FileNotFoundError("Place input.jpg in the same folder.")

# --- Laplacian Edge Detection ---
lap = cv2.Laplacian(gray, cv2.CV_64F)
lap_abs = cv2.convertScaleAbs(lap)

# --- Laplacian Sharpening ---
# Original + (Original - Blurred) = Original + Edges
sharpened = cv2.addWeighted(gray, 1.5, lap_abs, -0.5, 0)

# --- High-Pass Filter (Original - Blurred) ---
blur = cv2.GaussianBlur(gray, (5, 5), 0)
high_pass = cv2.subtract(gray, blur)
high_pass_abs = cv2.convertScaleAbs(high_pass)

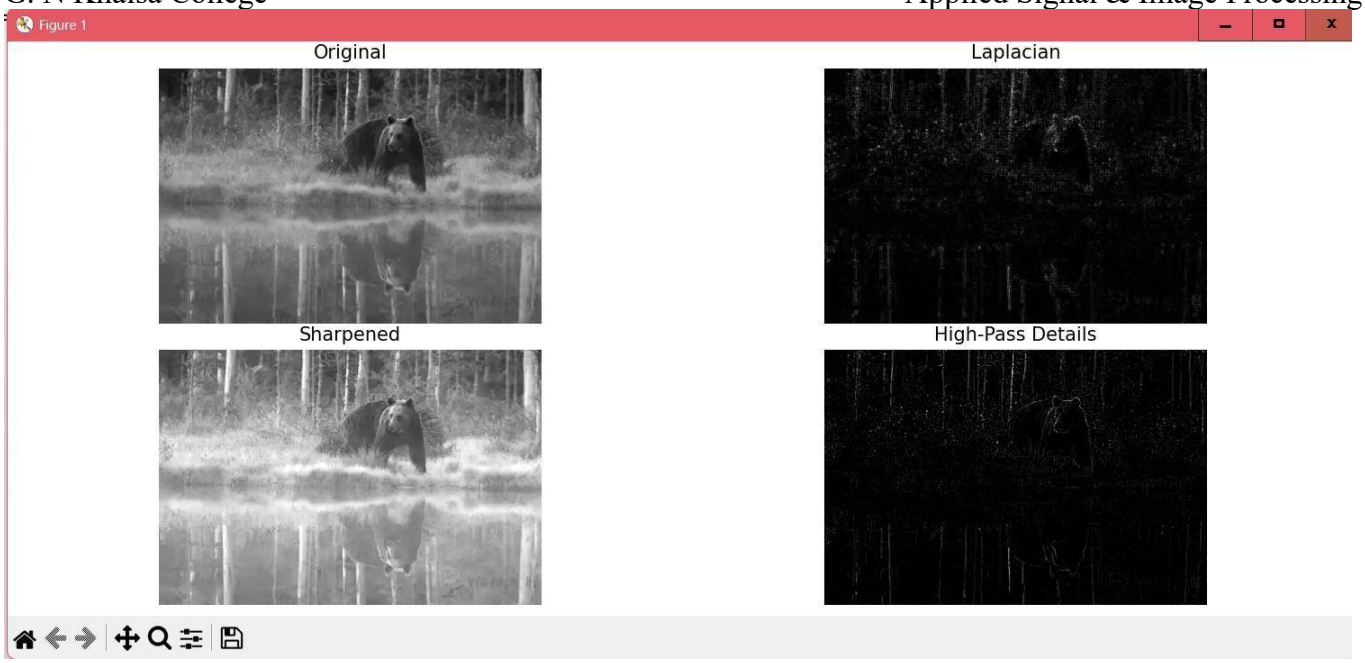
# --- Plotting ---
plt.figure(figsize=(12, 8))

images = [gray, lap_abs, sharpened, high_pass_abs]
titles = ["Original", "Laplacian", "Sharpened", "High-Pass Details"]

for i, (image, title) in enumerate(zip(images, titles), 1):
    plt.subplot(2, 2, i)
    plt.imshow(image, cmap="gray")
    plt.title(title)
    plt.axis("off")

plt.tight_layout()
plt.show()
```

Output:


Figure 1
Original
Laplacian
Sharpened
High-Pass Details

4C) To calculate image gradients using Sobel operator and Laplacian

Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

gray = cv2.imread("input.jpg", cv2.IMREAD_GRAYSCALE)

if gray is None:
    raise FileNotFoundError("Place input.jpg in the same folder.")

# --- Adding Gaussian Noise ---
np.random.seed(10)
noise = np.random.normal(0, 25, gray.shape)
noisy = np.uint8(np.clip(gray.astype(float) + noise, 0, 255))

# --- Function to compute gradients and Laplacian ---
def edge_results(image):
    gx = cv2.Sobel(image, cv2.CV_64F, 1, 0, ksize=3)  # Horizontal gradient
    gy = cv2.Sobel(image, cv2.CV_64F, 0, 1, ksize=3)  # Vertical gradient
    magnitude = np.sqrt(gx**2 + gy**2)                # Gradient magnitude
    lap = cv2.Laplacian(image, cv2.CV_64F)            # Laplacian
    return magnitude, np.abs(lap)

# --- Compute for clean and noisy images ---
grad_clean, lap_clean = edge_results(gray)
grad_noisy, lap_noisy = edge_results(noisy)

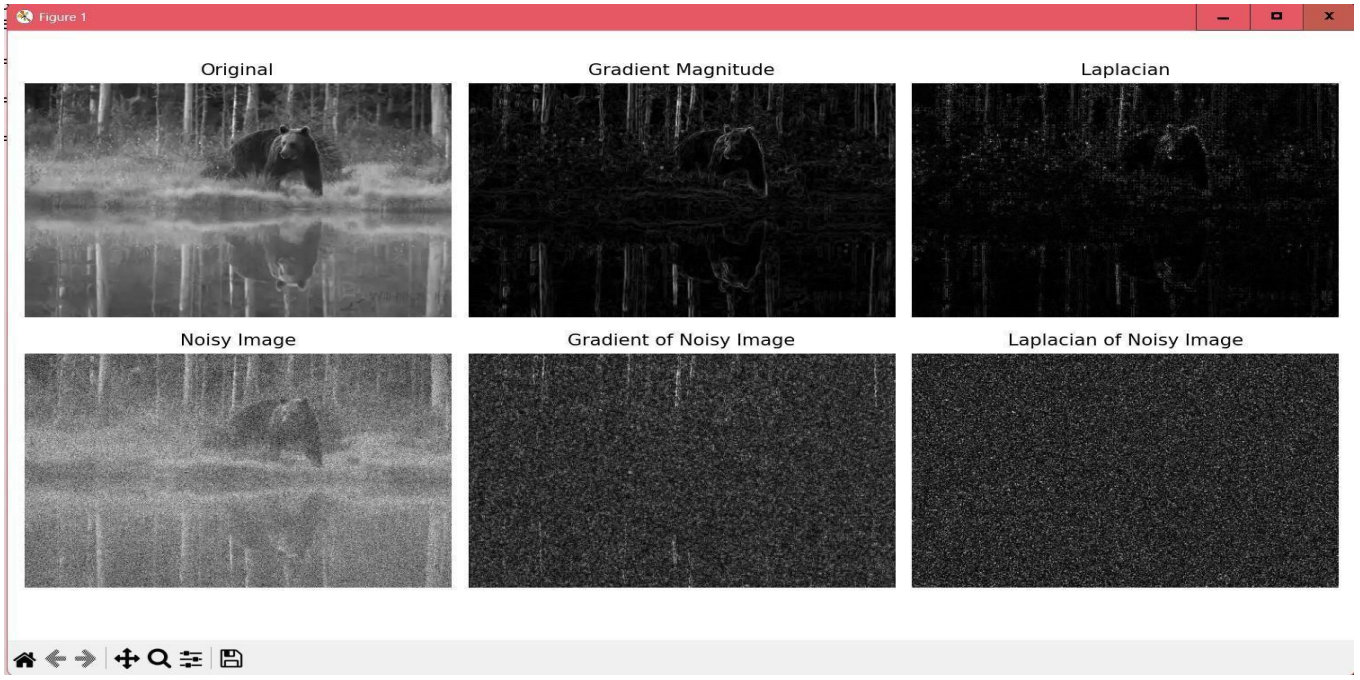
# --- Plotting ---
plt.figure(figsize=(12, 8))

images = [gray, grad_clean, lap_clean, noisy, grad_noisy, lap_noisy]
titles = [
    "Original", "Gradient Magnitude", "Laplacian",
    "Noisy Image", "Gradient of Noisy Image", "Laplacian of Noisy Image"
]
```

```
    for i, (image, title) in enumerate(zip(images, titles),
    1): plt.subplot(2, 3, i)
    plt.imshow(image, cmap="gray")
    plt.title(title)
    plt.axis("off")

plt.tight_layout()
plt.show()
```

Output:



Practical No: 01

Aim: Demonstrate Object-Oriented Databases

Q. Create the following types using OODBMS:

- **PersonType** (Per_id, Per_name, DOB)
- **BankType** (account_no, per_id, balance)

Queries

create or replace type persontype as object(personid number(3),person_name varchar2(20),dob date,member function age return number);

2 /

```
SQL> create or replace type persontype as object(personid number(3),person_name varchar2(20),dob date,member function age return number);
  2  /
```

create or replace type banktype as object(account_no number(20),person ref persontype,balance

number(20,2)); 2 /

```
SQL> create or replace type banktype as object(account_no number(20),person ref persontype,balance number(20,2));
  2  /

Type created.
```

Create the appropriate tables Insert 5 tuples and run the following queries:

create table person of

persontype; create table

banktable of banktype;

```
SQL> create table person of persontype;

Table created.

SQL> create table banktable of banktype;

Table created.
```

Insert values in the table

SQL> insert into person values(persontype(1,'sam','15-apr-2001')); SQL> insert into person values(persontype(1,'annie','14-apr-2012')); SQL> insert into person values(persontype(3,'marie','14-oct-2004')); SQL> insert into person values(persontype(4,'harry','24-oct-2005')); SQL> insert into person values(persontype(5,'sunny','14-jun-1993')); SQL> insert into person values(persontype(5,'sunny','14-jun-1993'));

```
SQL> insert into person values(persontype(1,'sam','15-apr-2001'));

1 row created.

SQL> insert into person values(persontype(1,'annie','14-apr-2012'));

1 row created.

SQL> insert into person values(persontype(3,'marie','14-oct-2004'));

1 row created.

SQL> insert into person values(persontype(4,'harry','24-oct-2005'));

1 row created.

SQL> insert into person values(persontype(5,'sunny','14-jun-1993'));

1 row created.
```

SQL> insert into banktable select 101,ref(p),100000 from person p where p.person_name = 'sam';
SQL> insert into banktable select 102,ref(p),20000 from person p where p.person_name = 'annie';
SQL> insert into banktable select 103,ref(p),30000 from person p where p.person_name = 'marie';
SQL> insert into banktable select 104,ref(p),40000 from person p where p.person_name = 'harry';
SQL> insert into banktable select 105,ref(p),50000 from person p where p.person_name = 'sunny';

```
SQL> insert into banktable select 101,ref(p),100000 from person p where p.person_name = 'sam';

1 row created.

SQL> insert into banktable select 102,ref(p),20000 from person p where p.person_name = 'annie';

1 row created.

SQL> insert into banktable select 103,ref(p),30000 from person p where p.person_name = 'marie';

1 row created.

SQL> insert into banktable select 104,ref(p),40000 from person p where p.person_name = 'harry';

1 row created.

SQL> insert into banktable select 105,ref(p),50000 from person p where p.person_name = 'sunny';

1 row created.
```

(i) Find name and age of a person

select p.person_name,p.age() "age" from person p;

```
SQL> select p.person_name,p.age() "age" from person p;

PERSON_NAME                age
-------------------- ----------
sam                          25
annie                        14
marie                        22
harry                        21
sunny                        33
```

(ii) Find all names with account number.

select p.person_name, b.account_no from banktable b ,person p where p.personid = b.person.personid;

```
SQL> select p.person_name, b.account_no from banktable b ,person p where p.personid = b.person.personid;

PERSON_NAME          ACCOUNT_NO
-------------------- ----------
sam                         101
sam                         102
annie                       101
annie                       102
marie                       103
harry                       104
sunny                       105

7 rows selected.
```

(iii) Find the names with balance more than 12,000

select p.person_name,b.balance from banktable b ,person p where p.personid = b.person.personid and b.balance > 12000;

```
SQL> select p.person_name,b.balance from banktable b ,person p where p.personid = b.person.personid and b.balance > 12000;

PERSON_NAME            BALANCE
-------------------- ----------
sam                     100000
sam                      20000
annie                   100000
annie                    20000
marie                    30000
harry                    40000
sunny                    50000

7 rows selected.
```

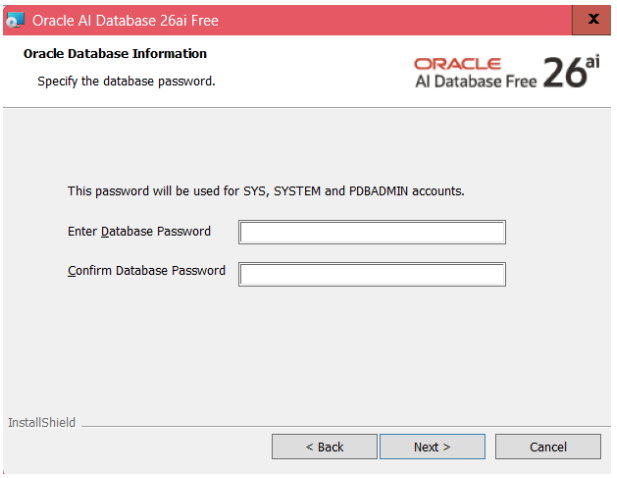
Practical No: 02

Aim: Demonstrate a distributed database environment by dividing a given global database table into

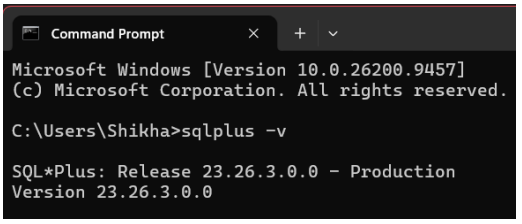
1. **Horizontal fragment**
2. **Vertical fragments**

1) Horizontal Fragmentation

Step 1: Go to Browser and Download Oracle 26ai and in get started section -> Extract the zip file -> Run setup.exe (Set username and password – system/system -> Open SQL Plus



Step 2: Open command prompt and type sqlplus -v to check if SQL Plus is installed or not



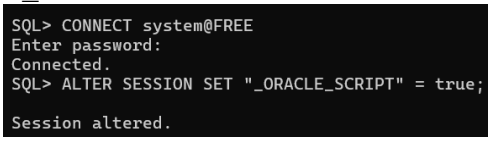
Step 3: Open SQL Plus and enter the username and password

Step 4: Implement the following commands:

A) Connect as SYSTEM and create users

Connect as system
CONNECT system@FREE
Enter password: system

Allow creation of users without C## prefix in Oracle 12c+
ALTER SESSION SET "_ORACLE_SCRIPT" = true;



Create users

CREATE USER user1 IDENTIFIED BY u1;
CREATE USER user2 IDENTIFIED BY u2;
CREATE USER user3 IDENTIFIED BY u3;

```
SQL> CREATE USER user1 IDENTIFIED BY u1;

User created.

SQL> CREATE USER user2 IDENTIFIED BY u2;

User created.

SQL> CREATE USER user3 IDENTIFIED BY u3;

User created.
```

Grant privileges

GRANT CREATE SESSION, CREATE TABLE, CREATE SEQUENCE, CREATE VIEW, CREATE PROCEDURE, CONNECT, RESOURCE, CREATE DATABASE LINK, UNLIMITED TABLESPACE TO user1;

GRANT CREATE SESSION, CREATE TABLE, CREATE SEQUENCE, CREATE VIEW, CREATE PROCEDURE, CONNECT, RESOURCE, CREATE DATABASE LINK, UNLIMITED TABLESPACE TO user2;

GRANT CREATE SESSION, CREATE TABLE, CREATE SEQUENCE, CREATE VIEW, CREATE PROCEDURE, CONNECT, RESOURCE, CREATE DATABASE LINK, UNLIMITED TABLESPACE TO user3;

```
SQL> GRANT CREATE SESSION, CREATE TABLE, CREATE SEQUENCE, CREATE VIEW,
  2        CREATE PROCEDURE, CONNECT, RESOURCE, CREATE DATABASE LINK,
  3        UNLIMITED TABLESPACE TO user1;

Grant succeeded.

SQL> GRANT CREATE SESSION, CREATE TABLE, CREATE SEQUENCE, CREATE VIEW,
  2        CREATE PROCEDURE, CONNECT, RESOURCE, CREATE DATABASE LINK,
  3        UNLIMITED TABLESPACE TO user2;

Grant succeeded.

SQL> GRANT CREATE SESSION, CREATE TABLE, CREATE SEQUENCE, CREATE VIEW,
  2        CREATE PROCEDURE, CONNECT, RESOURCE, CREATE DATABASE LINK,
  3        UNLIMITED TABLESPACE TO user3;

Grant succeeded.
```

B) Create the global table Emp and insert records

Still connected as system

```
CREATE TABLE emp (
   eno  NUMBER(3)  PRIMARY  KEY,
   ename  VARCHAR2(20),
   address  VARCHAR2(30),
   email    VARCHAR2(30),
   sal  NUMBER(6)
);
```

```
SQL> CREATE TABLE emp (
  2      eno     NUMBER(3) PRIMARY KEY,
  3      ename   VARCHAR2(20),
  4      address VARCHAR2(30),
  5      email   VARCHAR2(30),
  6      sal     NUMBER(6)
  7  );

Table created.
```

```
INSERT INTO emp VALUES (101, 'ram',  'dadar',  'ram@gmail.com',  10000);
INSERT INTO emp VALUES (102, 'tom',  'cst',     'tom@gmail.com',  25000);
INSERT INTO emp VALUES (103, 'sam',  'wadala',  'sam@gmail.com',  30000);
INSERT INTO emp VALUES (104, 'sonu',  'wadala',  'sonu@gmail.com',  12000);
INSERT INTO emp VALUES (105, 'monu',  'matunga', 'monu@gmail.com',  24000);
INSERT INTO emp VALUES (106, 'mona',  'sion',    'mona@gmail.com', 35000);
INSERT INTO emp VALUES (107, 'sona',  'sion',    'sona@gmail.com',  40000);
INSERT INTO emp VALUES (108, 'harry', 'kurla',   'harry@gmail.com', 30000);
INSERT INTO emp VALUES (109, 'marry', 'kurla',   'marry@gmail.com', 60000);
INSERT INTO emp VALUES (110, 'anni',  'kurla',  'anni@gmail.com',  10000);

COMMIT;
```

```
SQL> INSERT INTO emp VALUES (101, 'ram',   'dadar',   'ram@gmail.com',   10000);
1 row created.
SQL> INSERT INTO emp VALUES (102, 'tom',   'cst',     'tom@gmail.com',   25000);
1 row created.
SQL> INSERT INTO emp VALUES (103, 'sam',   'wadala',  'sam@gmail.com',   30000);
1 row created.
SQL> INSERT INTO emp VALUES (104, 'sonu',  'wadala',  'sonu@gmail.com',  12000);
1 row created.
SQL> INSERT INTO emp VALUES (105, 'monu',  'matunga', 'monu@gmail.com',  24000);
1 row created.
SQL> INSERT INTO emp VALUES (106, 'mona',  'sion',    'mona@gmail.com',  35000);
1 row created.
SQL> INSERT INTO emp VALUES (107, 'sona',  'sion',    'sona@gmail.com',  40000);
1 row created.
SQL> INSERT INTO emp VALUES (108, 'harry', 'kurla',   'harry@gmail.com', 30000);
1 row created.
SQL> INSERT INTO emp VALUES (109, 'marry', 'kurla',   'marry@gmail.com', 60000);
1 row created.
SQL> INSERT INTO emp VALUES (110, 'anni',  'kurla',   'anni@gmail.com',  10000);
1 row created.
SQL> COMMIT;
Commit complete.
```

C) Create fragment emp1 at USER1

Connect as user1
CONNECT user1@FREE
Password: u1

Create database link to global database (system schema)

CREATE DATABASE LINK l1
CONNECT TO system IDENTIFIED BY system
USING 'FREE';

```
SQL> CONNECT user1@FREE
Enter password:
Connected.
SQL> CREATE DATABASE LINK l1
  2  CONNECT TO system IDENTIFIED BY system
  3  USING 'FREE';

Database link created.
```

Create horizontal fragment emp1: salary <= 10000

CREATE TABLE emp1 AS
SELECT * FROM emp@l1
WHERE sal <= 10000;

ALTER TABLE emp1 ADD CONSTRAINT emp1_pk PRIMARY KEY (eno);

COMMIT;

```
SQL> CREATE TABLE emp1 AS
  2  SELECT * FROM emp@l1
  3  WHERE sal <= 10000;

Table created.

SQL> ALTER TABLE emp1 ADD CONSTRAINT emp1_pk PRIMARY KEY (eno);

Table altered.

SQL> COMMIT;

Commit complete.
```

D) Create fragment emp2 at USER2

Create horizontal fragment emp2: 10000 < salary <= 20000

```
CREATE TABLE emp2 AS
SELECT * FROM emp@l22
WHERE sal > 10000 AND sal <= 20000;

ALTER TABLE emp2 ADD CONSTRAINT emp2_pk PRIMARY KEY (eno);

COMMIT;
```

```
SQL> CREATE TABLE emp2 AS
  2  SELECT * FROM emp@l22
  3  WHERE sal > 10000 AND sal <= 20000;

Table created.

SQL> ALTER TABLE emp2 ADD CONSTRAINT emp2_pk PRIMARY KEY (eno);

Table altered.

SQL> COMMIT;

Commit complete.
```

E) Create database links in global site and consistency trigger

Connect back as system

```
CONNECT system@FREE

Links from global site to fragment sites
CREATE DATABASE LINK l1
CONNECT TO user1 IDENTIFIED BY u1
USING 'FREE';

CREATE DATABASE LINK l22
CONNECT TO user2 IDENTIFIED BY u2
USING 'FREE';
```

```
SQL> CONNECT system@FREE
Enter password:
Connected.
SQL> CREATE DATABASE LINK l1
  2  CONNECT TO user1 IDENTIFIED BY u1
  3  USING 'FREE';

Database link created.

SQL>
SQL> CREATE DATABASE LINK l22
  2  CONNECT TO user2 IDENTIFIED BY u2
  3  USING 'FREE';

Database link created.
```

Trigger to route new inserts to correct fragment

```
CREATE OR REPLACE TRIGGER trig_insert_emp
```

```
AFTER INSERT ON emp
FOR EACH ROW
BEGIN
   IF :NEW.sal <= 10000 THEN
      INSERT INTO emp1@l1
      VALUES (:NEW.eno, :NEW.ename, :NEW.address, :NEW.email, :NEW.sal);

   ELSIF :NEW.sal > 10000 AND :NEW.sal <= 20000 THEN
      INSERT INTO emp2@l22
      VALUES (:NEW.eno, :NEW.ename, :NEW.address, :NEW.email, :NEW.sal);

   ELSE
      NULL;
   END IF;
END;
/
```

```
SQL> CREATE OR REPLACE TRIGGER trig_insert_emp
  2  AFTER INSERT ON emp
  3  FOR EACH ROW
  4  BEGIN
  5      IF :NEW.sal <= 10000 THEN
  6          INSERT INTO emp1@l1
  7          VALUES (:NEW.eno, :NEW.ename, :NEW.address, :NEW.email, :NEW.sal);
  8
  9      ELSIF :NEW.sal > 10000 AND :NEW.sal <= 20000 THEN
 10          INSERT INTO emp2@l22
 11          VALUES (:NEW.eno, :NEW.ename, :NEW.address, :NEW.email, :NEW.sal);
 12
 13      ELSE
 14  NULL;
 15      END IF;
 16  END;
 17  /

Trigger created.
```

F) Test the trigger

Insert a new employee with salary 14500

```
INSERT INTO emp VALUES (111, 'ratan', 'bandra', 'ratan@gmail.com', 14500);
COMMIT;
```

Check whether it appears in emp2

```
SELECT * FROM emp2@l22;
```

```
SQL> INSERT INTO emp VALUES (111, 'ratan', 'bandra', 'ratan@gmail.com', 14500);

1 row created.

SQL> COMMIT;

Commit complete.

SQL> SELECT * FROM emp2@l22;

       ENO ENAME                ADDRESS
---------- -------------------- ------------------------------
EMAIL                                 SAL
------------------------------ ----------
       104 sonu                 wadala
sonu@gmail.com                       12000

       111 ratan                bandra
ratan@gmail.com                      14500
```

G) Fire the required queries from USER3

Connect as user3
CONNECT user3@FREE
Password: u3
Create database links

```
CREATE DATABASE LINK l0
CONNECT TO system IDENTIFIED BY system
USING 'FREE';

CREATE DATABASE LINK l1
CONNECT TO user1 IDENTIFIED BY u1
USING 'FREE';

CREATE DATABASE LINK l22
CONNECT TO user2 IDENTIFIED BY u2
USING 'FREE';
```

```
SQL> CONNECT user3@FREE
Enter password:
Connected.
SQL> CREATE DATABASE LINK l0
  2  CONNECT TO system IDENTIFIED BY system
  3  USING 'FREE';

Database link created.

SQL> CREATE DATABASE LINK l1
  2  CONNECT TO user1 IDENTIFIED BY u1
  3  USING 'FREE';

Database link created.

SQL> CREATE DATABASE LINK l22
  2  CONNECT TO user2 IDENTIFIED BY u2
  3  USING 'FREE';

Database link created.
```

(i) Find the salary of all employees

```
SELECT sal FROM emp@l0;
```

```
SQL> SELECT sal FROM emp@l0;

       SAL
----------
     10000
     25000
     30000
     12000
     24000
     35000
     40000
     30000
     60000
     10000
     14500

11 rows selected.
```

(ii) Find the Email of all employees where salary = 12,000

```
SELECT email FROM emp2@l22 WHERE sal = 12000;
```

```
SQL> SELECT email FROM emp2@l22 WHERE sal = 12000;

EMAIL
----------------------------
sonu@gmail.com
```

(iii) Find the employee name and Email where employee number is known

SELECT ename, email FROM emp@l0 WHERE eno = 109;

```
SQL> SELECT ename, email FROM emp@l0 WHERE eno = 109;

ENAME                EMAIL
-------------------- ------------------------------
marry                marry@gmail.com
```

(iv) Find the employee name where salary > 20,000

SELECT ename FROM emp@l0 WHERE sal > 20000;

```
SQL> SELECT ename FROM emp@l0 WHERE sal > 20000;

ENAME
--------------------
tom
sam
monu
mona
sona
harry
marry

7 rows selected.
```

2) **Perform Vertical Fragmentation** on a global conceptual schema Emp(Eno, Ename, Address, email, Salary) and insert 10 records. Divide Emp into vertical fragments Emp1(Eno, Ename, Address) and Emp2(Eno, Email, Salary) on two different nodes. Fire the following queries:
a) Find the salary of an employee where employee number is known.
b) Find the Email where the employee name is known.
c) Find the employee name and email where employee number is known.
d) Find the employee name whose salary is > 2000.

SIDE NOTE: Oracle Connection Error?

If you get ORA-12541 (No Listener) or a login failure:

1. **Command Prompt:** Type lsnrctl status to check. If stopped, type lsnrctl start.
2. **Windows Services:** Press Win + R → type services.msc → Enter. Start Oracle...TNSListener and OracleServiceFREE (or XE).
3. **College Username –** system , Password - tycs

TAB 1 — SYSTEM (Global Database)

Log in to SQLPlus as SYSTEM.*

Step 1: Enable normal user creation

ALTER SESSION SET "_ORACLE_SCRIPT"=true;

Step 2: Create the users

(If user already exists then use the command - DROP USER user1 CASCADE;)
CREATE USER user1 IDENTIFIED BY u1;
CREATE USER user2 IDENTIFIED BY u2;
CREATE USER user3 IDENTIFIED BY u3;

```
SQL> CREATE USER user1 IDENTIFIED BY u1;

User created.

SQL> CREATE USER user2 IDENTIFIED BY u2;

User created.

SQL> CREATE USER user3 IDENTIFIED BY u3;

User created.
```

Step 3: Grant privileges to all users

GRANT CREATE SESSION, CREATE TABLE, CREATE SEQUENCE, CREATE VIEW, CREATE PROCEDURE, CONNECT, RESOURCE, CREATE DATABASE LINK, UNLIMITED TABLESPACE TO user1;
GRANT CREATE SESSION, CREATE TABLE, CREATE SEQUENCE, CREATE VIEW, CREATE PROCEDURE, CONNECT, RESOURCE, CREATE DATABASE LINK, UNLIMITED TABLESPACE TO user2;
GRANT CREATE SESSION, CREATE TABLE, CREATE SEQUENCE, CREATE VIEW, CREATE PROCEDURE, CONNECT, RESOURCE, CREATE DATABASE LINK, UNLIMITED TABLESPACE TO user3;

```
SQL> GRANT CREATE SESSION, CREATE TABLE, CREATE SEQUENCE, CREATE VIEW,
CREATE PROCEDURE, CONNECT, RESOURCE, CREATE DATABASE LINK, UNLIMITED TA
BLESPACE TO user1;

Grant succeeded.

SQL> GRANT CREATE SESSION, CREATE TABLE, CREATE SEQUENCE, CREATE VIEW,
CREATE PROCEDURE, CONNECT, RESOURCE, CREATE DATABASE LINK, UNLIMITED TA
BLESPACE TO user2;

Grant succeeded.

SQL> GRANT CREATE SESSION, CREATE TABLE, CREATE SEQUENCE, CREATE VIEW,
CREATE PROCEDURE, CONNECT, RESOURCE, CREATE DATABASE LINK, UNLIMITED TA
BLESPACE TO user3;

Grant succeeded.
```

Step 4: Create the global conceptual schema table

```
CREATE TABLE emp(
  eno  NUMBER(3)  PRIMARY  KEY,
  ename  VARCHAR2(20),
  address  VARCHAR2(30),
  email    VARCHAR2(30),
  sal  NUMBER(6)
);
```

```
SQL> CREATE TABLE emp(
  2    eno     NUMBER(3) PRIMARY KEY,
  3    ename   VARCHAR2(20),
  4    address VARCHAR2(30),
  5    email   VARCHAR2(30),
  6    sal     NUMBER(6)
  7  );

Table created.
```

Step 5: Insert 10 records

```
INSERT INTO emp VALUES (101,'ram','dadar','ram@gmail.com',10000);
INSERT INTO emp VALUES (102,'tom','cst','tom@gmail.com',25000);
INSERT INTO emp VALUES (103,'sam','wadala','sam@gmail.com',30000);
INSERT INTO emp VALUES (104,'sonu','wadala','sonu@gmail.com',12000);
INSERT INTO emp VALUES (105,'monu','matunga','monu@gmail.com',24000);
INSERT INTO emp VALUES (106,'mona','sion','mona@gmail.com',35000);
INSERT INTO emp VALUES (107,'sona','sion','sona@gmail.com',40000);
INSERT INTO emp VALUES (108,'harry','kurla','harry@gmail.com',30000);
INSERT INTO emp VALUES (109,'marry','kurla','marry@gmail.com',60000);
INSERT INTO emp VALUES (110,'anni','kurla','anni@gmail.com',10000);
```

Step 6: Commit and verify

```
COMMIT;
SELECT * FROM emp;
```

```
SQL> SELECT * FROM emp;

       ENO ENAME                ADDRESS
---------- -------------------- ---------------
EMAIL                                 SAL
------------------------------ ----------
       101 ram                  dadar
ram@gmail.com                        10000

       102 tom                  cst
tom@gmail.com                        25000

       103 sam                  wadala
sam@gmail.com                        30000
```

TAB 2 — USER1 (Fragment Emp1)

Open a brand new SQLPlus window. Type the following:
Connect as user1 (No 'USER' keyword!)
CONNECT user1@FREE
Enter password: u1

Create database link to global schema
CREATE DATABASE LINK l1 CONNECT TO system IDENTIFIED BY tycs USING 'FREE';

Create the vertical fragment Emp1(Eno, Ename, Address)
CREATE TABLE e1 AS SELECT eno, ename, address FROM emp@l1;

Commit and verify
COMMIT;
SELECT * FROM e1;

```
SQL> CREATE DATABASE LINK l1 CONNECT TO system IDENTIFIED BY system USING 'FREE';

Database link created.

SQL> CREATE TABLE e1 AS SELECT eno, ename, address FROM emp@l1;

Table created.

SQL> COMMIT;

Commit complete.

SQL> SELECT * FROM e1;

       ENO ENAME                ADDRESS
---------- -------------------- ------------------------------
       101 ram                  dadar
       102 tom                  cst
       103 sam                  wadala
       104 sonu                 wadala
       105 monu                 matunga
       106 mona                 sion
       107 sona                 sion
       108 harry                kurla
       109 marry                kurla
       110 anni                 kurla

10 rows selected.
```

TAB 3 — USER2 (Fragment Emp2)

Open a third SQLPlus window. Type the following:
Connect as user1 (No 'USER' keyword!)
CONNECT user1@FREE
Enter password: u1

Create database link to global schema

CREATE DATABASE LINK l1 CONNECT TO system IDENTIFIED BY tycs USING 'FREE';

Create the vertical fragment Emp1(Eno, Ename, Address)
CREATE TABLE e1 AS SELECT eno, ename, address FROM emp@l1;

Commit and verify
COMMIT;
SELECT * FROM e1;

TAB 3 — USER2 (Fragment Emp2)

Open a third SQLPlus window. Type the following:
Connect as user2
CONNECT user2@FREE
Enter password: u2

Create database link to global schema
CREATE DATABASE LINK l2 CONNECT TO system IDENTIFIED BY tycs USING 'FREE';

Create the vertical fragment Emp2(Eno, Email, Salary)
CREATE TABLE e2 AS SELECT eno, email, sal FROM emp@l2;
Commit and verify
COMMIT;
SELECT * FROM e2;

```
SQL> CREATE DATABASE LINK l2 CONNECT TO system IDENTIFIED BY system USING 'FREE';

Database link created.

SQL> CREATE TABLE e2 AS SELECT eno, email, sal FROM emp@l2;

Table created.

SQL> COMMIT
  2  ;

Commit complete.

SQL> SELECT * FROM e2;

       ENO EMAIL                              SAL
---------- ------------------------------ ----------
       101 ram@gmail.com                       10000
       102 tom@gmail.com                       25000
       103 sam@gmail.com                       30000
       104 sonu@gmail.com                      12000
       105 monu@gmail.com                      24000
       106 mona@gmail.com                      35000
       107 sona@gmail.com                      40000
       108 harry@gmail.com                     30000
       109 marry@gmail.com                     60000
       110 anni@gmail.com                      10000

10 rows selected.
```

TAB 4 — USER3 (Queries on Fragments)

Open a fourth SQLPlus window. Type the following:
Connect as user3
CONNECT user3@FREE
Enter password: u3

Create links to User1 and User2

CREATE DATABASE LINK l11 CONNECT TO user1 IDENTIFIED BY u1 USING 'FREE';
CREATE DATABASE LINK l22 CONNECT TO user2 IDENTIFIED BY u2 USING 'FREE';

```
SQL> CONNECT user3@FREE
Enter password:
Connected.
SQL> CREATE DATABASE LINK l11 CONNECT TO user1 IDENTIFIED BY u1 USING 'FREE';

Database link created.

SQL> CREATE DATABASE LINK l22 CONNECT TO user2 IDENTIFIED BY u2 USING 'FREE';

Database link created.
```

Query a) Find the salary of an employee where employee number is known.

SELECT sal FROM e2@l22 WHERE eno=&eno;

```
SQL> SELECT sal FROM e2@l22 WHERE eno=&eno;
Enter value for eno: 105
old   1: SELECT sal FROM e2@l22 WHERE eno=&eno
new   1: SELECT sal FROM e2@l22 WHERE eno=105

       SAL
----------
     24000
```

Query b) Find the Email where the employee name is known.

SELECT email FROM e2@l22 WHERE eno=(SELECT eno FROM e1@l11 WHERE ename='&ename');

```
SQL> SELECT email FROM e2@l22 WHERE eno=(SELECT eno FROM e1@l11 WHERE ename='&ename');
Enter value for ename: mona
old   1: SELECT email FROM e2@l22 WHERE eno=(SELECT eno FROM e1@l11 WHERE ename='&ename')
new   1: SELECT email FROM e2@l22 WHERE eno=(SELECT eno FROM e1@l11 WHERE ename='mona')

EMAIL
------------------------------
mona@gmail.com
```

Query c) Find the employee name and email where employee number is known.

SELECT e1.ename, e2.email
FROM e1@l11 e1, e2@l22 e2
WHERE e1.eno=e2.eno AND e1.eno=&eno;

```
SQL> SELECT e1.ename, e2.email
  2  FROM e1@l11 e1, e2@l22 e2
  3  WHERE e1.eno=e2.eno AND e1.eno=&eno;
Enter value for eno: 101
old   3: WHERE e1.eno=e2.eno AND e1.eno=&eno
new   3: WHERE e1.eno=e2.eno AND e1.eno=101

ENAME                EMAIL
-------------------- ------------------------------
ram                  ram@gmail.com
```

Query d) Find the employee name whose salary is > 2000.

SELECT ename FROM e1@l11 WHERE eno IN (SELECT eno FROM e2@l22 WHERE sal>2000);

Practical No: 03

Aim: Perform the CRUD operations using

1. **MongoDB**
2. **CouchDB**

1) MongoDB

Step 1: Download MongoDB Server, MongoDB Shell (mongosh) / Compass

Step 2: Open the terminal and type mongosh to start the connection (Error: Check Environment Variables)

Step 3: Start with the implementation

1) Create Database and Collection

use CollegeDB
db.createCollection("students")

```
test> use CollegeDB
switched to db CollegeDB
CollegeDB> db.createCollection("students")
{ ok: 1 }
```

2) Insert at least 5 student documents

db.students.insertMany([

{_id: 1, name: "Rahul", age: 22, department: "Computer Science", semester: 2, city: "Mumbai", marks: 85, skills: ["Java", "MongoDB", "Python"]},

{_id: 2, name: "Priya", age: 21, department: "Computer Science", semester: 2, city: "Delhi", marks: 78, skills: ["C++", "MongoDB"]},

{_id: 3, name: "Amit", age: 23, department: "IT", semester: 4, city: "Pune", marks: 92, skills: ["Python", "SQL"]},

{_id: 4, name: "Sneha", age: 20, department: "AI", semester: 2, city: "Bangalore", marks: 88, skills: ["Python", "Machine Learning", "MongoDB"]},

{_id: 5, name: "Vikram", age: 24, department: "IT", semester: 6, city: "Chennai", marks: 35, skills: ["Java"]},

{_id: 6, name: "Ananya", age: 22, department: "AI", semester: 4, city: "Hyderabad", marks: 95, skills: ["Deep Learning", "MongoDB"]}

])

```
CollegeDB> db.students.insertMany([
|   {_id: 1, name: "Rahul", age: 22, department: "Computer Science", semester: 2, city: "Mumbai", marks: 85, skills: ["Java", "MongoDB", "Python
"]},
|   {_id: 2, name: "Priya", age: 21, department: "Computer Science", semester: 2, city: "Delhi", marks: 78, skills: ["C++", "MongoDB"]},
|   {_id: 3, name: "Amit", age: 23, department: "IT", semester: 4, city: "Pune", marks: 92, skills: ["Python", "SQL"]},
|   {_id: 4, name: "Sneha", age: 20, department: "AI", semester: 2, city: "Bangalore", marks: 88, skills: ["Python", "Machine Learning", "MongoD
B"]},
|   {_id: 5, name: "Vikram", age: 24, department: "IT", semester: 6, city: "Chennai", marks: 35, skills: ["Java"]},
|   {_id: 6, name: "Ananya", age: 22, department: "AI", semester: 4, city: "Hyderabad", marks: 95, skills: ["Deep Learning", "MongoDB"]}
| ])
{
  acknowledged: true,
  insertedIds: { '0': 1, '1': 2, '2': 3, '3': 4, '4': 5, '5': 6 }
}
```

3) Display all students belonging to the Computer Science department.

db.students.find({ department: "Computer Science" })

```
CollegeDB> db.students.find({ department: "Computer Science" })
[
  {
    _id: 1,
    name: 'Rahul',
    age: 22,
    department: 'Computer Science',
    semester: 2,
    city: 'Mumbai',
    marks: 85,
    skills: [ 'Java', 'MongoDB', 'Python' ]
  },
  {
    _id: 2,
    name: 'Priya',
    age: 21,
    department: 'Computer Science',
    semester: 2,
    city: 'Delhi',
    marks: 78,
    skills: [ 'C++', 'MongoDB' ]
  }
]
```

4) Find all students whose marks are greater than 80 and who belong to Semester 2

db.students.find({ marks: { $gt: 80 }, semester: 2 })

```
CollegeDB> db.students.find({ marks: { $gt: 80 }, semester: 2 })
[
  {
    _id: 1,
    name: 'Rahul',
    age: 22,
    department: 'Computer Science',
    semester: 2,
    city: 'Mumbai',
    marks: 85,
    skills: [ 'Java', 'MongoDB', 'Python' ]
  },
  {
    _id: 4,
    name: 'Sneha',
    age: 20,
    department: 'AI',
    semester: 2,
    city: 'Bangalore',
    marks: 88,
    skills: [ 'Python', 'Machine Learning', 'MongoDB' ]
  }
]
```

5) Display only the name, department, and marks of all students without displaying the _id field

db.students.find({}, { _id: 0, name: 1, department: 1, marks: 1 })

```
CollegeDB> db.students.find({}, { _id: 0, name: 1, department: 1, marks: 1 })
[
  { name: 'Rahul', department: 'Computer Science', marks: 85 },
  { name: 'Priya', department: 'Computer Science', marks: 78 },
  { name: 'Amit', department: 'IT', marks: 92 },
  { name: 'Sneha', department: 'AI', marks: 88 },
  { name: 'Vikram', department: 'IT', marks: 35 },
  { name: 'Ananya', department: 'AI', marks: 95 }
]
```

6) Display all students sorted by marks in descending order

db.students.find().sort({ marks: -1 })

```
CollegeDB> db.students.find().sort({ marks: -1 })
[
  {
    _id: 6,
    name: 'Ananya',
    age: 22,
    department: 'AI',
    semester: 4,
    city: 'Hyderabad',
    marks: 95,
    skills: [ 'Deep Learning', 'MongoDB' ]
  },
  {
    _id: 3,
    name: 'Amit',
    age: 23,
    department: 'IT',
    semester: 4,
    city: 'Pune',
    marks: 92,
    skills: [ 'Python', 'SQL' ]
  },
  {
    _id: 4,
    name: 'Sneha',
    age: 20,
    department: 'AI',
    semester: 2,
    city: 'Bangalore',
    marks: 88,
    skills: [ 'Python', 'Machine Learning', 'MongoDB' ]
  },
```

7) Find all students who have "MongoDB" as one of their skills

db.students.find({ skills: "MongoDB" })

```
CollegeDB> db.students.find({ skills: "MongoDB" })
[
  {
    _id: 1,
    name: 'Rahul',
    age: 22,
    department: 'Computer Science',
    semester: 2,
    city: 'Mumbai',
    marks: 85,
    skills: [ 'Java', 'MongoDB', 'Python' ]
  },
  {
    _id: 2,
    name: 'Priya',
    age: 21,
    department: 'Computer Science',
    semester: 2,
    city: 'Delhi',
    marks: 78,
    skills: [ 'C++', 'MongoDB' ]
  },
  {
    _id: 4,
    name: 'Sneha',
    age: 20,
    department: 'AI',
    semester: 2,
    city: 'Bangalore',
    marks: 88,
    skills: [ 'Python', 'Machine Learning', 'MongoDB' ]
  },
```

8) Increase all students' marks by 5

db.students.updateMany({}, { $inc: { marks: 5 } })

```
CollegeDB> db.students.updateMany({}, { $inc: { marks: 5 } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 6,
  modifiedCount: 6,
  upsertedCount: 0
}
```

9) Delete all student records whose marks are less than 40

db.students.deleteMany({ marks: { $lt: 40 } })

```
CollegeDB> db.students.deleteMany({ marks: { $lt: 40 } })
{ acknowledged: true, deletedCount: 0 }
```

10) Using the Aggregation Framework, calculate the average marks of students department-wise

db.students.aggregate([

{ $group: { _id: "$department", averageMarks: { $avg: "$marks" } } }

])

```
CollegeDB> db.students.aggregate([
|   { $group: { _id: "$department", averageMarks: { $avg: "$marks" } } }
| ])
[
  { _id: 'Computer Science', averageMarks: 86.5 },
  { _id: 'AI', averageMarks: 96.5 },
  { _id: 'IT', averageMarks: 68.5 }
]
```

11) Write an aggregation query to display the number of students in each department

```
db.students.aggregate([
  { $group: { _id: "$department", totalStudents: { $sum: 1 } } }
])
```

```
CollegeDB> db.students.aggregate([
|   { $group: { _id: "$department", totalStudents: { $sum: 1 } } }
| ])
[
  { _id: 'IT', totalStudents: 2 },
  { _id: 'AI', totalStudents: 2 },
  { _id: 'Computer Science', totalStudents: 2 }
]
```

12) Find the top 3 highest-scoring students and display only their name, department, and marks

```
db.students.find({}, { _id: 0, name: 1, department: 1, marks: 1 }).sort({ marks: -1 }).limit(3)
```

```
CollegeDB> db.students.find({}, { _id: 0, name: 1, department: 1, marks: 1 }).sort({ marks: -1 }).limit(3)
[
  { name: 'Ananya', department: 'AI', marks: 100 },
  { name: 'Amit', department: 'IT', marks: 97 },
  { name: 'Sneha', department: 'AI', marks: 93 }
]
```

Indexing

Step 1: Open the terminal and type mongosh

Step 2: Perform the following queries

1) Setup and Data Insertion

```
use CollegeDB
db.students.insertMany([
  { _id: 1, name: "Rahul", department: "Computer Science", email: "rahul@gmail.com", marks: 85, semester: 2 },
  { _id: 2, name: "Priya", department: "Computer Science", email: "priya@gmail.com", marks: 78, semester: 2 },
  { _id: 3, name: "Amit", department: "IT", email: "amit@gmail.com", marks: 92, semester: 4 },
  { _id: 4, name: "Sneha", department: "AI", email: "sneha@gmail.com", marks: 88, semester: 2 },
  { _id: 5, name: "Vikram", department: "IT", email: "vikram@gmail.com", marks: 35, semester: 6 }
])
```

```
CollegeDB> db.students.insertMany([
|   { _id: 1, name: "Rahul", department: "Computer Science", email: "rahul@gmail.com", marks: 85, semester: 2 },
|   { _id: 2, name: "Priya", department: "Computer Science", email: "priya@gmail.com", marks: 78, semester: 2 },
|   { _id: 3, name: "Amit", department: "IT", email: "amit@gmail.com", marks: 92, semester: 4 },
|   { _id: 4, name: "Sneha", department: "AI", email: "sneha@gmail.com", marks: 88, semester: 2 },
|   { _id: 5, name: "Vikram", department: "IT", email: "vikram@gmail.com", marks: 35, semester: 6 }
| ])
{
  acknowledged: true,
  insertedIds: { '0': 1, '1': 2, '2': 3, '3': 4, '4': 5 }
}
```

2) Check Indexes Before Creation

db.students.getIndexes()

```
CollegeDB> db.students.getIndexes()
[ { v: 2, key: { _id: 1 }, name: '_id_' } ]
```

Explanation: Lists all existing indexes on the collection. By default, MongoDB only creates an index on _id

3) Measure Performance BEFORE Indexing (Collection Scan)

db.students.find({ email: "rahul@gmail.com" }).explain("executionStats")

```
CollegeDB> db.students.find({ email: "rahul@gmail.com" }).explain("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'CollegeDB.students',
    parsedQuery: { email: { '$eq': 'rahul@gmail.com' } },
    indexFilterSet: false,
    queryHash: 'ED28E3D2',
    planCacheShapeHash: 'ED28E3D2',
    planCacheKey: '30FCCA9A',
    optimizationTimeMillis: 9,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'COLLSCAN',
      filter: { email: { '$eq': 'rahul@gmail.com' } },
      direction: 'forward'
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 1,
    executionTimeMillis: 14,
    totalKeysExamined: 0,
    totalDocsExamined: 5,
    executionStages: {
      isCached: false,
      stage: 'COLLSCAN',
      filter: { email: { '$eq': 'rahul@gmail.com' } },
      nReturned: 1,
      executionTimeMillisEstimate: 0,
      works: 6,
```

Explanation: Runs the query and provides detailed execution stats. Because there is no index on email, MongoDB must scan every single document to find a match.

Output (Key Metrics):
winningPlan.stage: "COLLSCAN" (Collection Scan)
totalDocsExamined: 5 (Scanned all 5 documents)
totalKeysExamined: 0 (No index keys used)
executionTimeMillis: 8 (Takes time to scan)

4) Create an Index

db.students.createIndex({ "email": 1 })

```
CollegeDB> db.students.createIndex({ "email": 1 })
email_1
```

Explanation: Creates a single-field ascending index on the email field to speed up future email searches.

5) Check Indexes After Creation

db.students.getIndexes()

```
CollegeDB> db.students.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { email: 1 }, name: 'email_1' }
]
```

6) Measure Performance AFTER Indexing (Index Scan)

db.students.find({ email: "rahul@gmail.com" }).explain("executionStats")

```
CollegeDB> db.students.find({ email: "rahul@gmail.com" }).explain("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'CollegeDB.students',
    parsedQuery: { email: { '$eq': 'rahul@gmail.com' } },
    indexFilterSet: false,
    queryHash: 'ED28E3D2',
    planCacheShapeHash: 'ED28E3D2',
    planCacheKey: '1E950542',
    optimizationTimeMillis: 17,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'FETCH',
      inputStage: {
        stage: 'IXSCAN',
        keyPattern: { email: 1 },
        indexName: 'email_1',
        isMultiKey: false,
        multiKeyPaths: { email: [] },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: { email: [ '["rahul@gmail.com", "rahul@gmail.com"]' ] }
      }
```

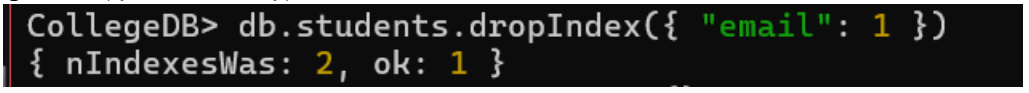


Explanation: Runs the exact same query again. MongoDB now uses the index to instantly locate the document, drastically reducing the amount of data it needs to read.

Output (Key Metrics):
winningPlan.stage: "FETCH" (Fetches the document)
inputStage.stage: "IXSCAN" (Index Scan - fast)
totalDocsExamined: 1 (Only 1 document scanned)
totalKeysExamined: 1 (Used 1 index key)
executionTimeMillis: 0 (Instant execution)

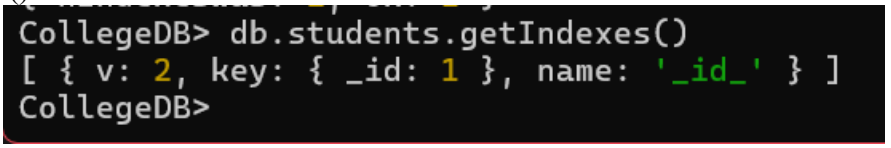
7) Drop or Delete the Index

db.students.dropIndex({ "email": 1 })



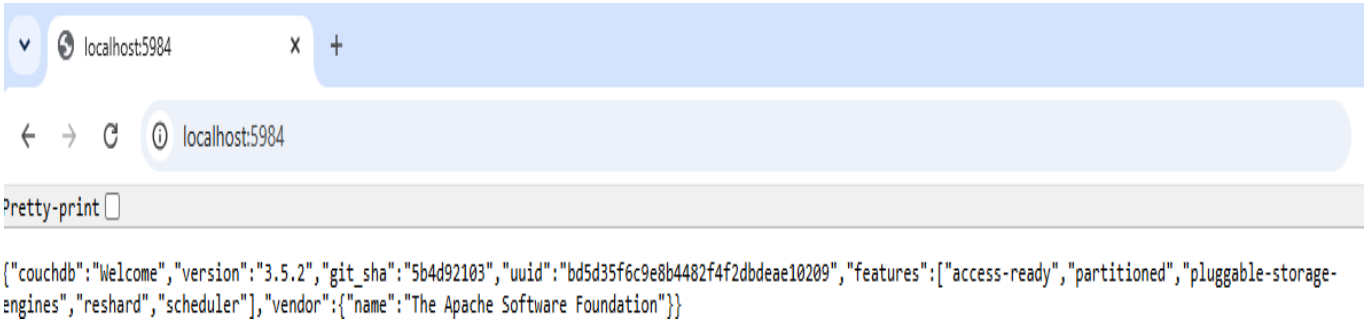
8) Verify the Index has been Dropped

db.students.getIndexes()

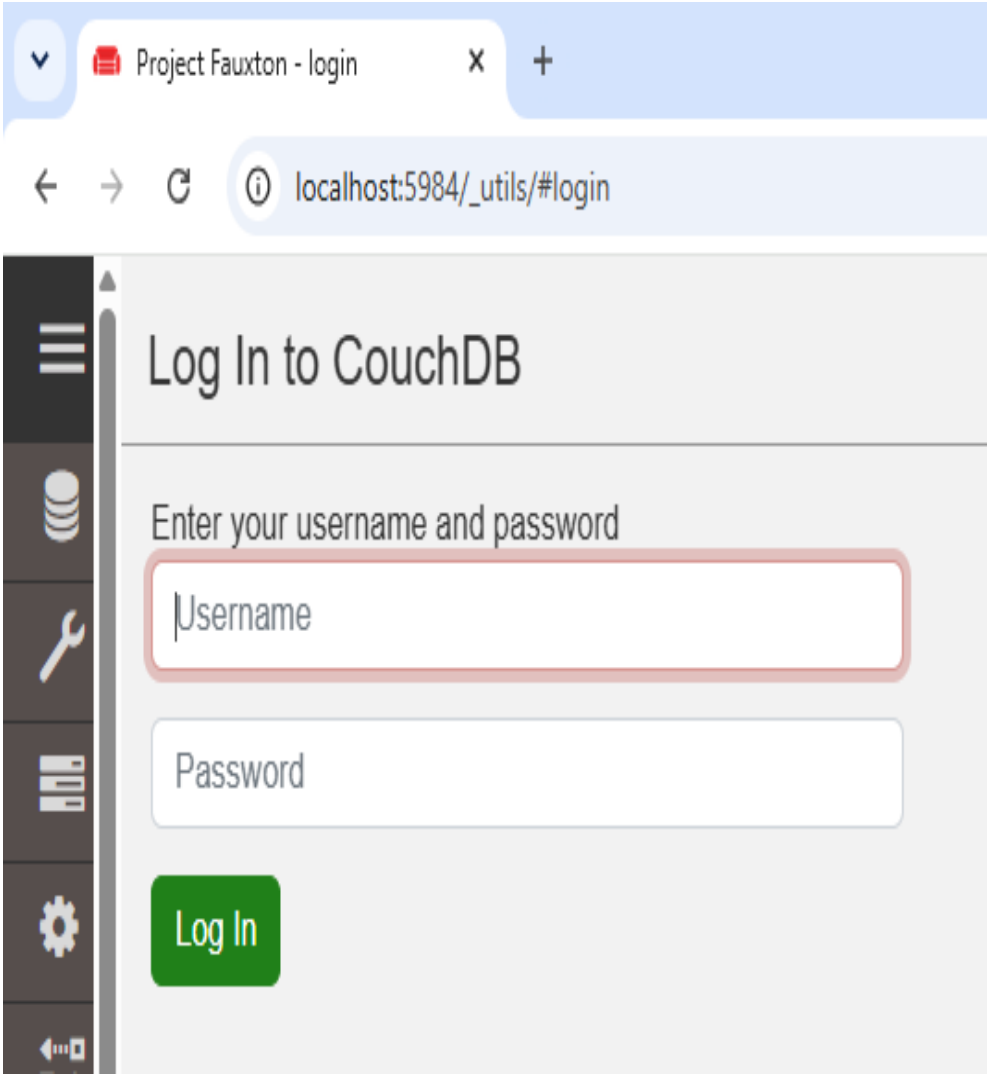


2) CouchDB

Step 1) Downloaded the Windows .msi installer from the official Apache CouchDB website
Run the installer set an Username - admin and Password - 123 and chose Single Node setup
After installation opened a web browser and go to the CouchDB welcome page http://localhost:5984

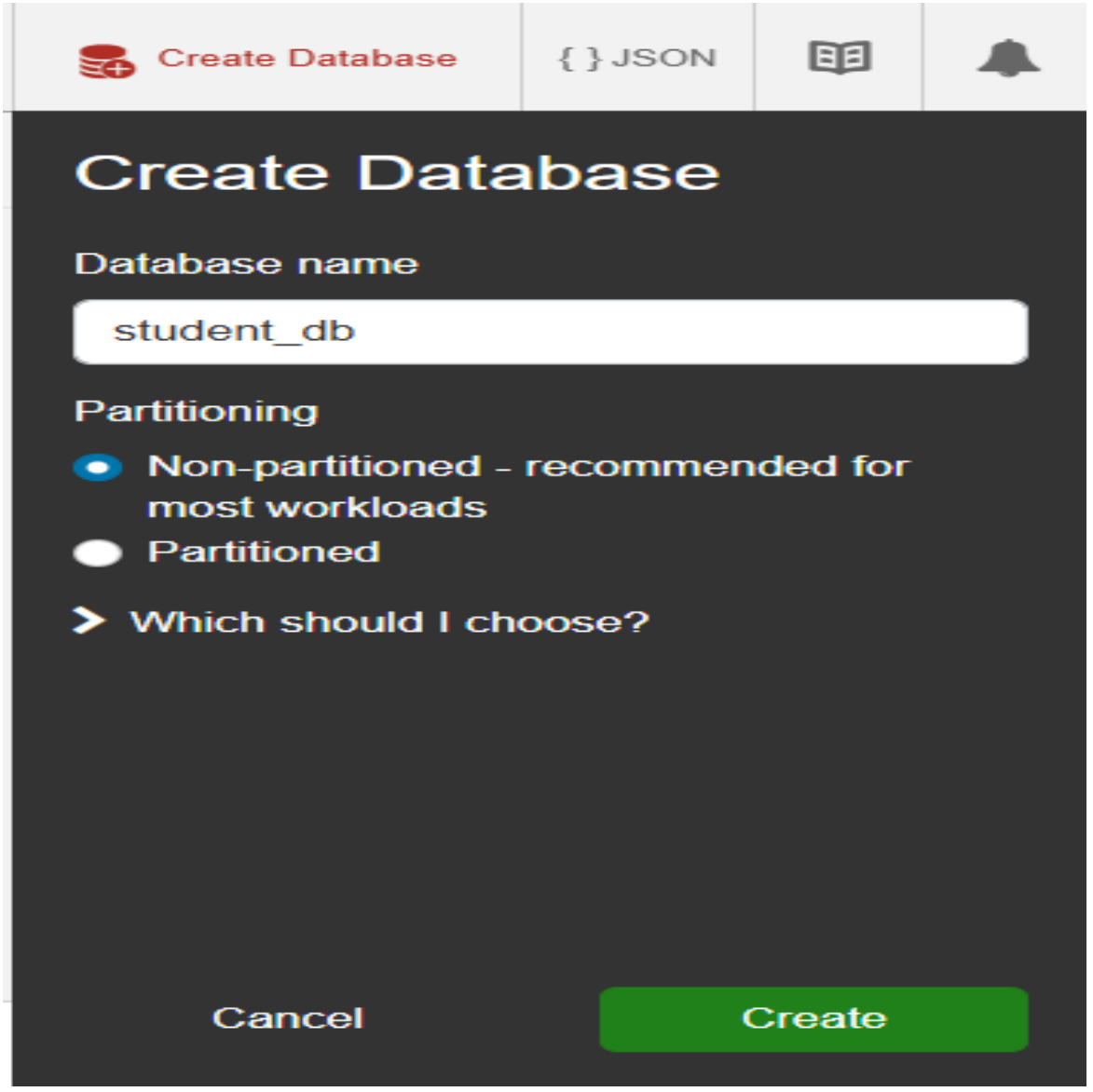


Opened the administration interface:
http://localhost:5984/_utils



Step 2: Create Database
In the interface look at the top right corner
Clicked the Create Database button

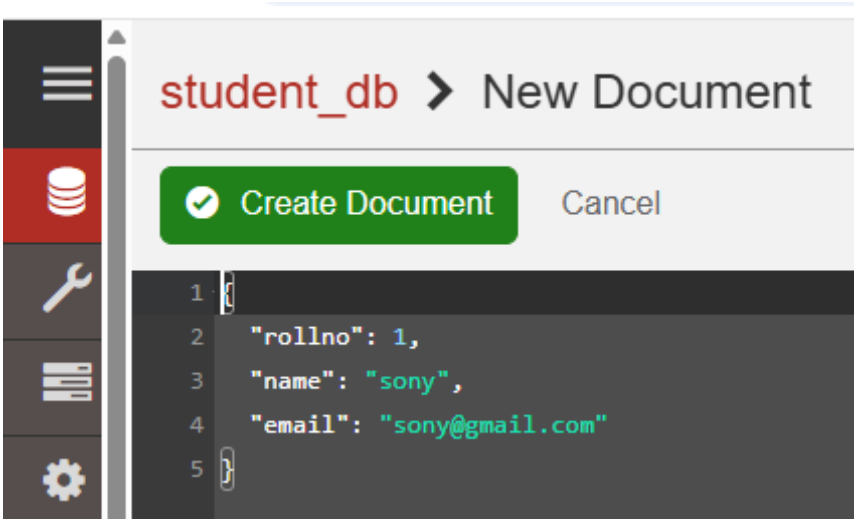
Typed the database name student_db and clicked Create



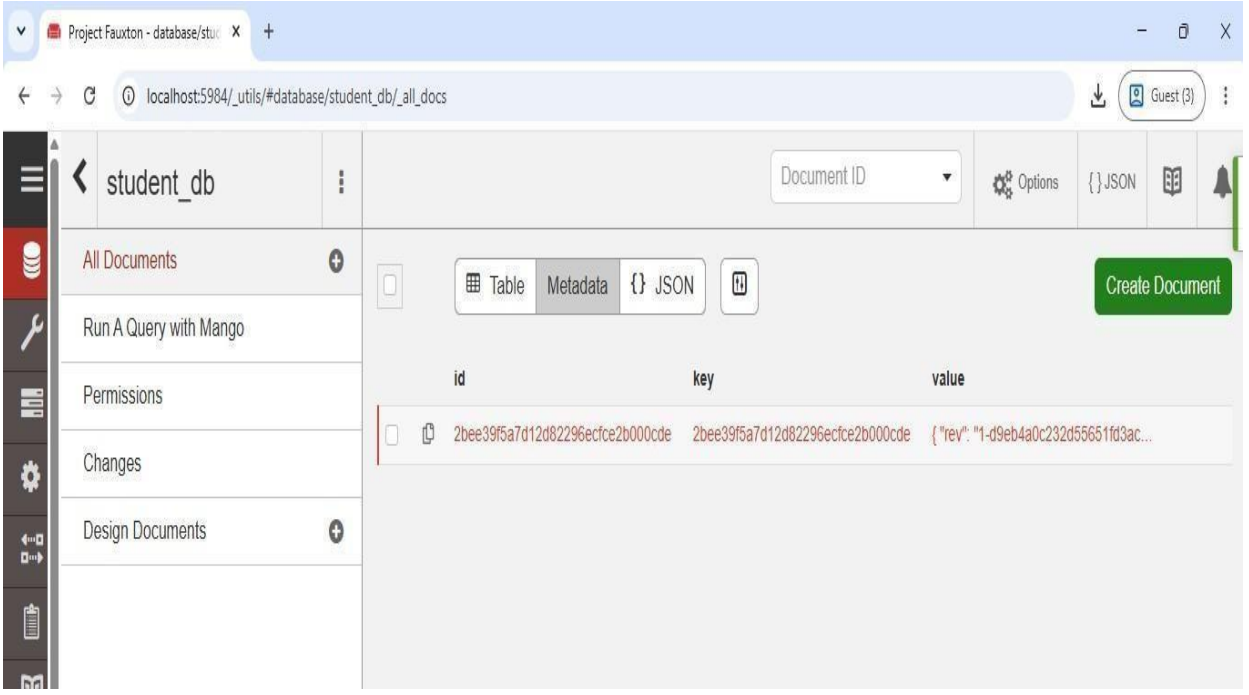
Step 3: Insert Data (Create Document)

Inside the student_db database, looked at the top right corner
Clicked the Create Document button
In the JSON editor, replaced the default {} with the following data: json
{

"rollno": 1,
"name": "sony",
"email"sony@gmail.com"
}

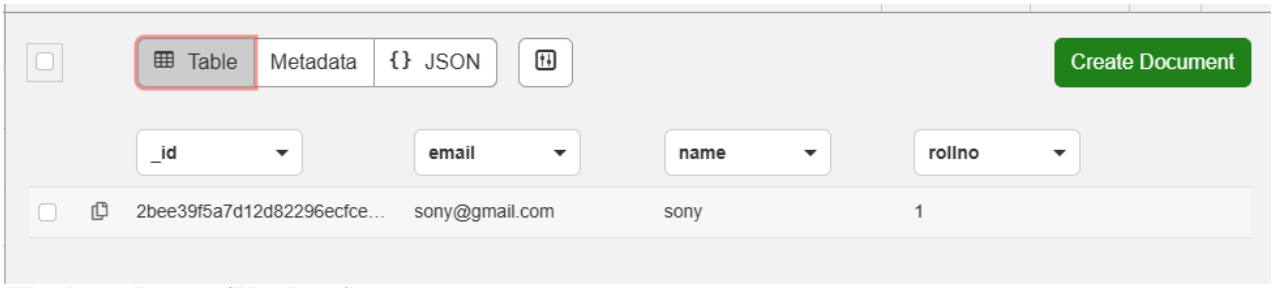


Clicked Create Document to save it



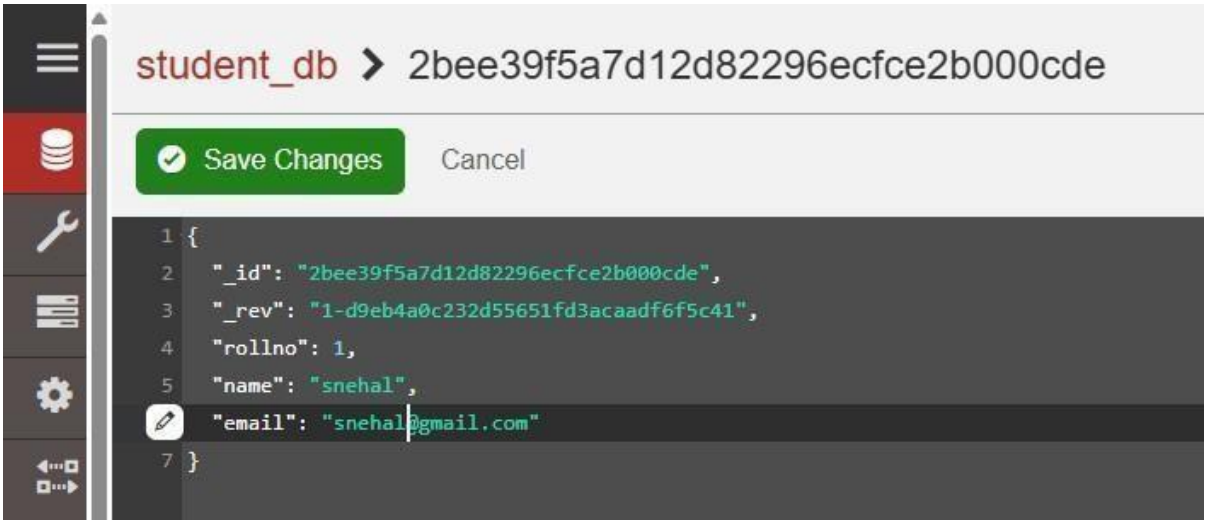
Step 4: View Data (Read)

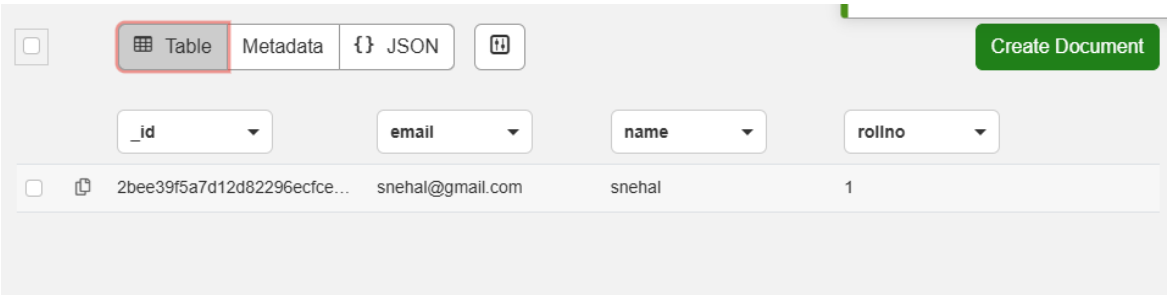
Clicked on All Documents in the left sidebar
Switched the view to Table View (using the toggle option in the top middle part) to see the data in a structured format



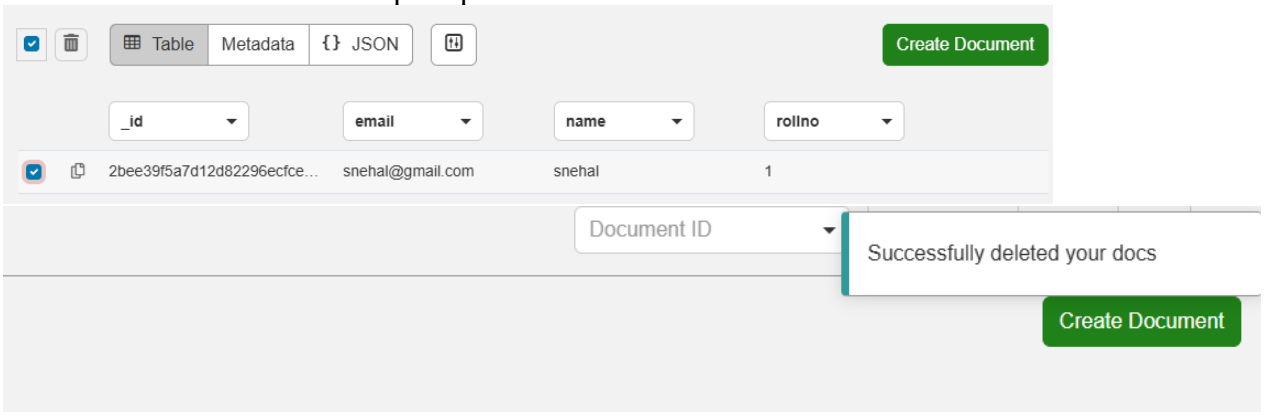
Step 5: Update Data (Update)

Clicked on the document to open it
Edited the JSON data. For example, changed "name": "sony" to "name": "snehal"
Clicked the Save Changes button (located at the top left corner)





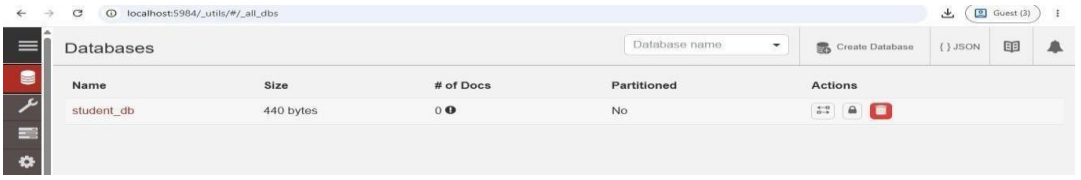
Step 6: Delete Data (Delete Document)
Returned to Table View (All Documents)
Checked the checkbox on the left side of the row to be deleted
Clicked the Delete icon (trash can) at the top
Confirmed the deletion on the prompt

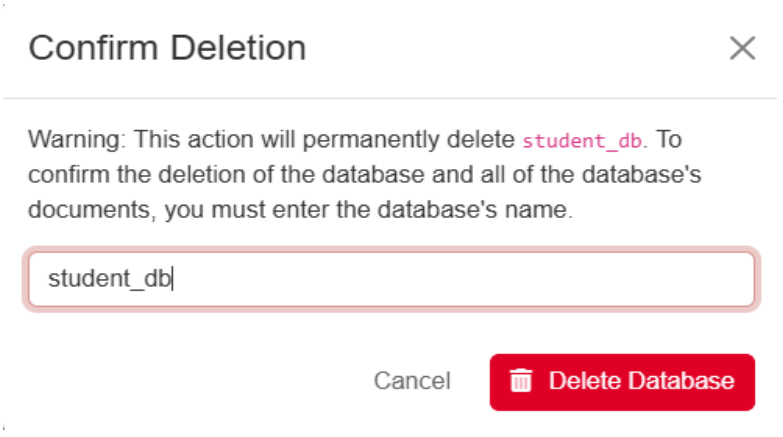


Step 7: Delete Database

Clicked the Database icon on the far left sidebar to return to the main database list
Located student_db in the list
Clicked the Delete icon (trash can) next to the database name

In the confirmation prompt, typed student_db and clicked Delete







Practical No: 04

Aim: Perform the CRUD operations using

1. **Redis**
2. **Apache Cassandra**

Part A – Redis

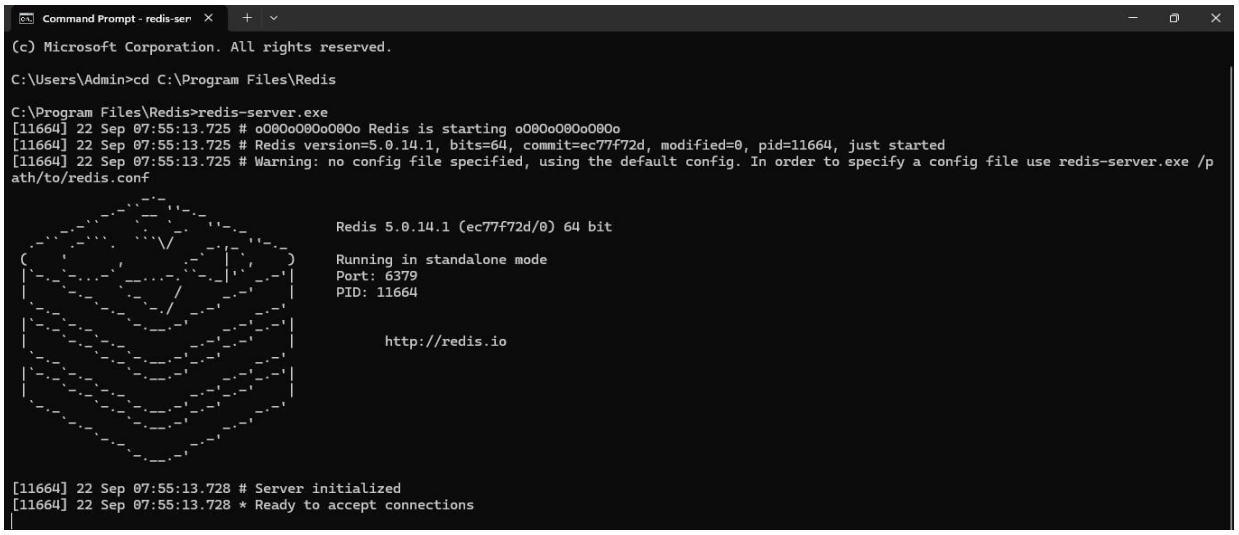
Step 1: Installation and Starting the Server

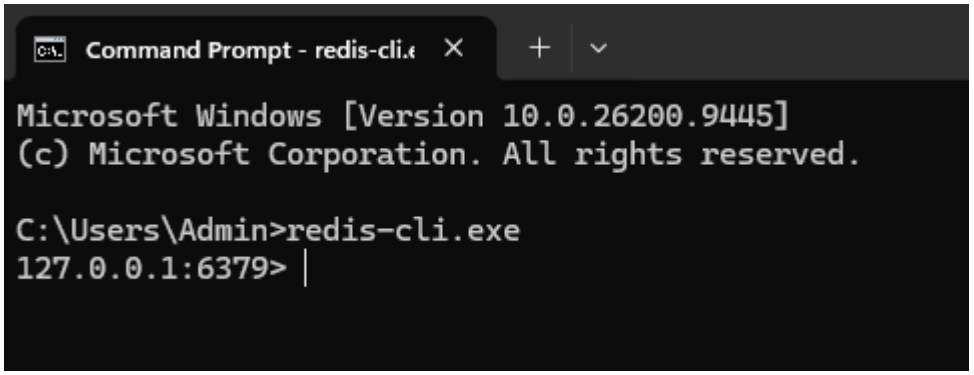
1. Downloaded and extracted the Redis zip file to a directory (e.g., `E:\redis`)
2. Opened Command Prompt and navigated to the Redis directory
3. Started the Redis Server by running redis-server.exe
4. Opened a second Command Prompt window and started the Redis Client by running redis-cli.exe
5. Tested the connection using the PING command Commands

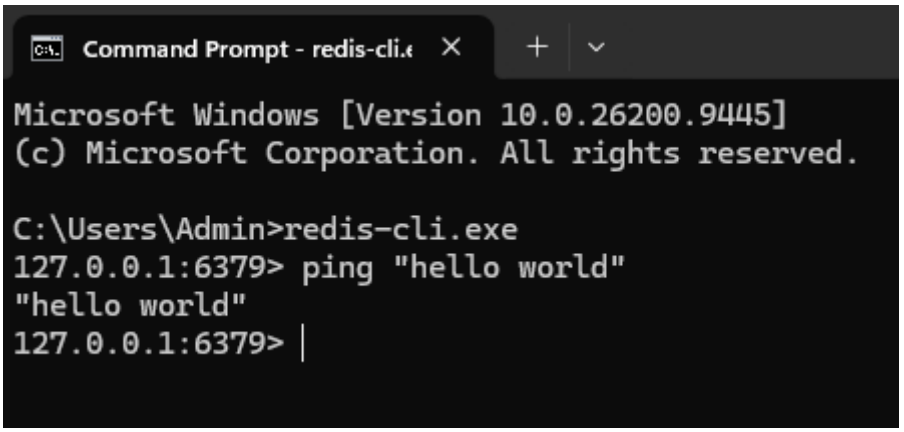
redis

ping

ping "hello world"







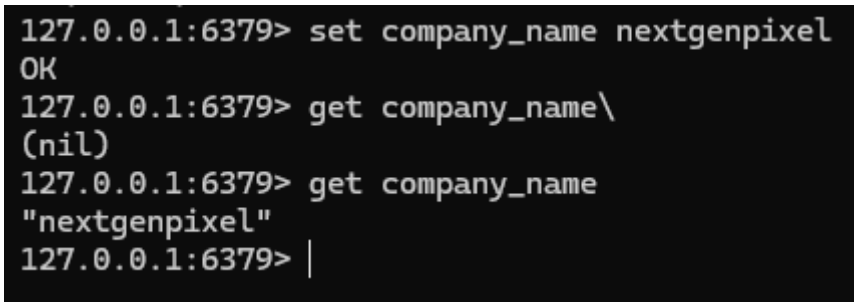
Step 2: Redis Strings (SET, GET, EXPIRE, APPEND, STRLEN)

Strings are the most basic data type in Redis we performed operations to add, view, and manipulate string values

Commands redis

set company_name nextgenpixel

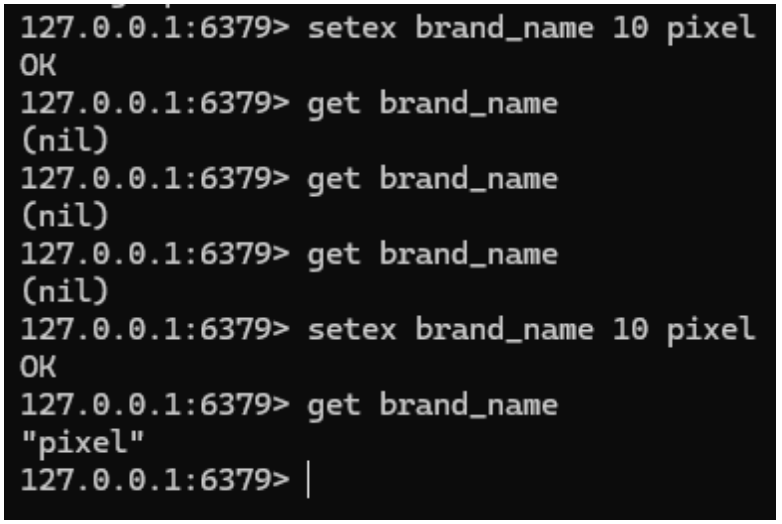
get company_name



setex brand_name 10

pixel get brand_name

get brand_name



set company_id 22

keys *

```
127.0.0.1:6379> set companyid 22
OK
127.0.0.1:6379> keys*
(error) ERR unknown command `keys*`, with args beginning with:
127.0.0.1:6379> keys *
1) "companyid"
2) "company_name"
127.0.0.1:6379>
```

del name

keys *

```
127.0.0.1:6379> del name
(integer) 0
127.0.0.1:6379> keys *
1) "companyid"
2) "company_name"
127.0.0.1:6379>
```

ttl head

expire head 10

```
127.0.0.1:6379> ttl head
(integer) -2
127.0.0.1:6379> expire head 10
(integer) 0
127.0.0.1:6379>
```

set phone_no 92959252952

expire phone_no 10

keys *

```
127.0.0.1:6379> set phone_no 1236547898
OK
127.0.0.1:6379> expire phone_no 10
(integer) 1
127.0.0.1:6379> keys *
1) "companyid"
2) "company_name"
3) "phone_no"
127.0.0.1:6379>
```

set address mumbai EX 5

get address

```
127.0.0.1:6379> set address mumbai EX 5
OK
127.0.0.1:6379> get address
(nil)
127.0.0.1:6379> set address mumbai EX 10
OK
127.0.0.1:6379> get address
"mumbai"
127.0.0.1:6379>
```

append company_name .co.in

get company_name

```
127.0.0.1:6379> append company_name .co.in
(integer) 18
127.0.0.1:6379> get company_name
"nextgenpixel.co.in"
127.0.0.1:6379>
```

append company_name website

get company_name

```
127.0.0.1:6379> append company_name website
(integer) 25
127.0.0.1:6379> get company_name
"nextgenpixel.co.inwebsite"
127.0.0.1:6379>
```

strlen company_name

```
127.0.0.1:6379> strlen company_name
(integer) 25
127.0.0.1:6379>
```

del companyid

keys *

```
127.0.0.1:6379> del companyid
(integer) 1
127.0.0.1:6379> keys *
1) "company_name"
127.0.0.1:6379>
```

flushall

keys *

```
127.0.0.1:6379> flushall
OK
127.0.0.1:6379> keys *
(empty list or set)
127.0.0.1:6379>
```

Step 3: Redis Lists (LPUSH, RPUSH, LRANGE, LPOP, RPOP, BLPOP)

Redis Lists are simply lists of strings, sorted by insertion order elements can be added to the head or the tail of the list

Commands

redis

LPUSH company_name nextgenpixel

LPUSH company_name adobe

LPUSH company_name frolikpixel

LRANGE company_name 0 10

```
127.0.0.1:6379> LPUSH company_name nextgenpixel
(integer) 1
127.0.0.1:6379> LPUSH company_name adobe
(integer) 2
127.0.0.1:6379> LPUSH company_name frolikpixel
(integer) 3
127.0.0.1:6379> LRANGE company_name 0 10
1) "frolikpixel"
2) "adobe"
3) "nextgenpixel"
127.0.0.1:6379>
```

LPUSH brand_name manyavar centric xbox

LRANGE brand_name 0 10

```
127.0.0.1:6379> LPUSH brand_name manyavar centric xbox
(integer) 3
127.0.0.1:6379> LRANGE brand_name 0 10
1) "xbox"
2) "centric"
3) "manyavar"
127.0.0.1:6379>
```

RPUSH brand_name web_engage

LRANGE brand_name 0 10

```
127.0.0.1:6379> RPUSH brand_name web_engage
(integer) 4
127.0.0.1:6379> LRANGE brand_name 0 10
1) "xbox"
2) "centric"
3) "manyavar"
4) "web_engage"
127.0.0.1:6379>
```

LINDEX brand_name 2

```
127.0.0.1:6379> LINDEX brand_name 2
"manyavar"
127.0.0.1:6379>
```

LSET brand_name 1 xbox360

LRANGE brand_name 0 10

```
127.0.0.1:6379> LSET brand_name 1 xbox360
OK
127.0.0.1:6379> LRANGE brand_name 0 10
1) "xbox"
2) "xbox360"
3) "manyavar"
4) "web_engage"
127.0.0.1:6379>
```

LPOP brand_name

LRANGE brand_name 0 10

```
127.0.0.1:6379> LPOP brand_name
"xbox"
127.0.0.1:6379> LRANGE brand_name 0 10
1) "xbox360"
2) "manyavar"
3) "web_engage"
```

RPOP brand_name

LRANGE brand_name 0 10

```
127.0.0.1:6379> RPOP brand_name
"web_engage"
127.0.0.1:6379> LRANGE brand_name 0 10
1) "xbox360"
2) "manyavar"
```

BLPOP brand_name 5

LRANGE brand_name 0 10

```
127.0.0.1:6379> BLPOP brand_name 5
1) "brand_name"
2) "xbox360"
127.0.0.1:6379> LRANGE brand_name 0 10
1) "manyavar"
```

LPUSH brand_name blubirch samsung

LRANGE brand_name 0 10

```
127.0.0.1:6379> LPUSH brand_name blubirch samsung
(integer) 3
127.0.0.1:6379> LRANGE brand_name 0 10
1) "samsung"
2) "blubirch"
3) "manyavar"
127.0.0.1:6379>
```

Step 4: Redis Sets (SADD, SMEMBERS, SCARD, SISMEMBER, SREM)

Redis Sets are an unordered collection of unique strings we performed operations to add, view, and remove members from sets

Commands

redis

SADD company_name bluebirchentaix biretail zentaix

SMEMBERS company_name

SCARD company_name

In Redis, a single key can only hold one type of data.

Earlier in the Lists section, you ran LPUSH company_name .This made the key company_name a List.

Now you are trying to run SADD company_name , which is for Sets. Redis is saying: "I can't do Set operations on a List!"

```
127.0.0.1:6379> SADD company_name bluebirchentaix biretail zentaix
(error) WRONGTYPE Operation against a key holding the wrong kind of value
127.0.0.1:6379> del company_name
(integer) 1
127.0.0.1:6379> SADD company_name bluebirchentaix biretail zentaix
(integer) 3
127.0.0.1:6379> SMEMBERS company_name
1) "zentaix"
2) "biretail"
3) "bluebirchentaix"
127.0.0.1:6379> SCARD company_name
(integer) 3
127.0.0.1:6379>
```

SADD brand_name posiflex
SADD brand_name dinglix
SADD brand_name spacex
SADD brand_name valience

```
127.0.0.1:6379> SADD brand_name posiflex
(error) WRONGTYPE Operation against a key holding the wrong kind of value
127.0.0.1:6379> del brand_name
(integer) 1
127.0.0.1:6379> SADD brand_name posiflex
(integer) 1
127.0.0.1:6379> SADD brand_name dinglix
(integer) 1
127.0.0.1:6379> SADD brand_name spacex
(integer) 1
127.0.0.1:6379> SADD brand_name valience
(integer) 1
127.0.0.1:6379>
```

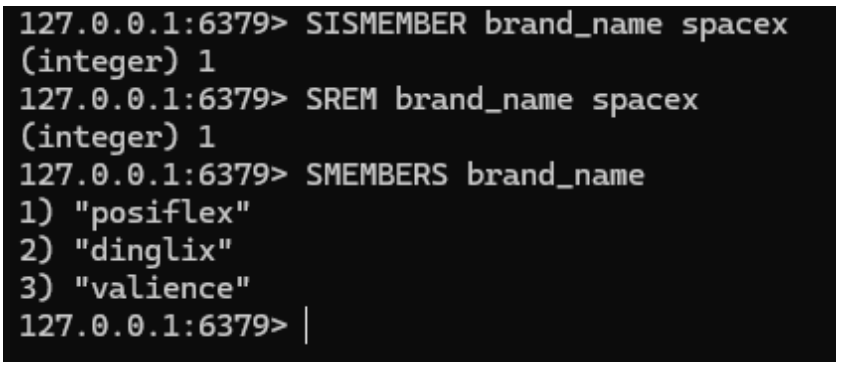
SMEMBERS brand_name
SCARD brand_name

```
127.0.0.1:6379> SMEMBERS brand_name
1) "posiflex"
2) "dinglix"
3) "valience"
4) "spacex"
127.0.0.1:6379> SCARD brand_name
(integer) 4
127.0.0.1:6379>
```

SISMEMBER brand_name spacex

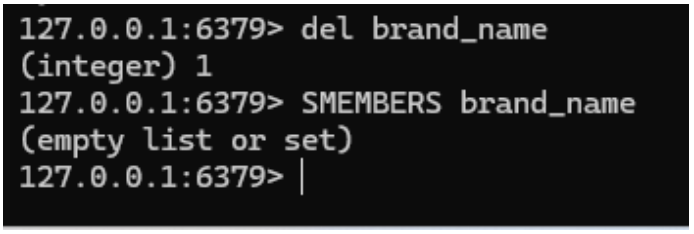
SREM brand_name spacex

SMEMBERS brand_name



del brand_name

SMEMBERS brand_name



Part B – Apache Cassandra

Step 1) To start the Apache Cassandra -> Open Command Prompt window and launched the CQL shell: cmd

cqlsh

Showing the Cassandra server starting in one window and the `cqlsh` prompt connected in the second window

Step 2: Keyspace Operations

A keyspace is an object that holds column families (tables). It is similar to a database in RDBMS.

CREATE KEYSPACE cscorner

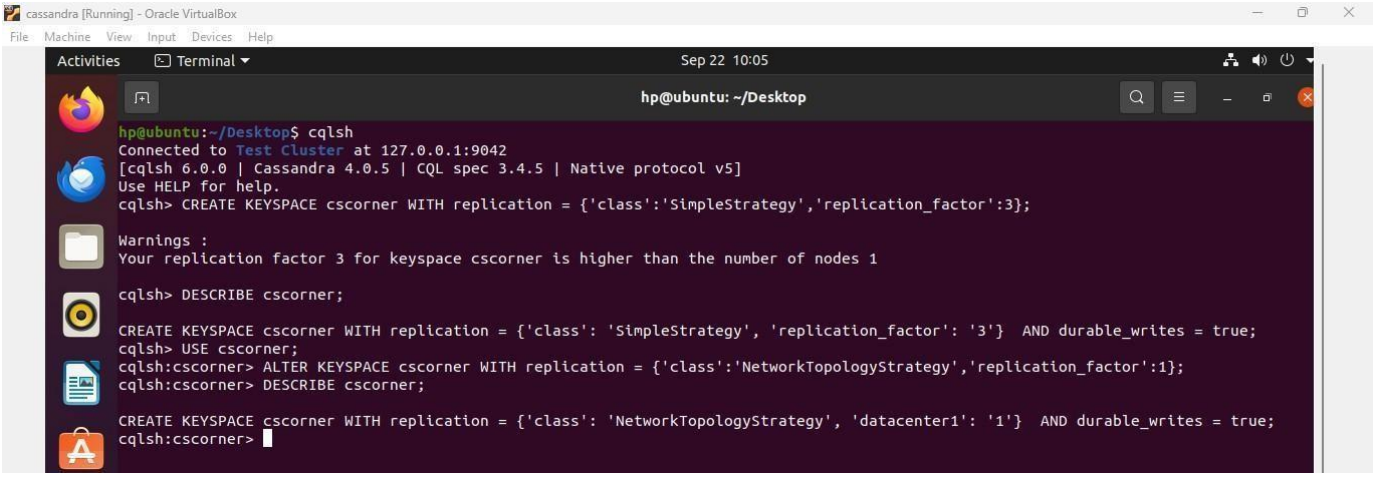
WITH replication = {'class':'SimpleStrategy', 'replication_factor':3};

DESCRIBE cscorner;

USE cscorner;

ALTER KEYSPACE cscorner

WITH replication = {'class':'NetworkTopologyStrategy', 'replication_factor':1}; Showing the successful creation and description of the `cscorner` keyspace



Step 3: Create Table and Insert Data (Create)

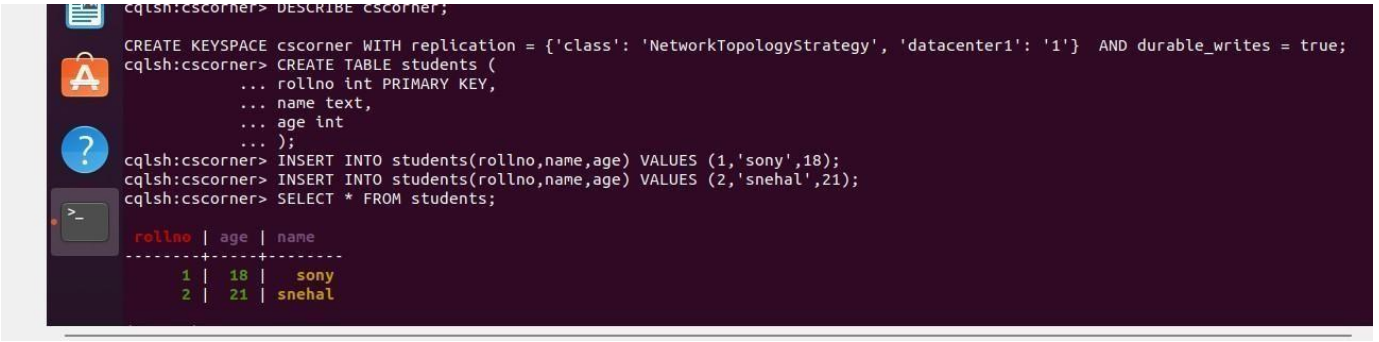
We created a table named `students` and inserted records into it.

CREATE TABLE students (rollno int PRIMARY KEY, name text,age int);

INSERT INTO students(rollno, name, age) VALUES (1, 'sunita', 18);

INSERT INTO students(rollno, name, age) VALUES (2, 'code with sunita', 20);

SELECT * FROM students;



Step 4: Update Data (Update)

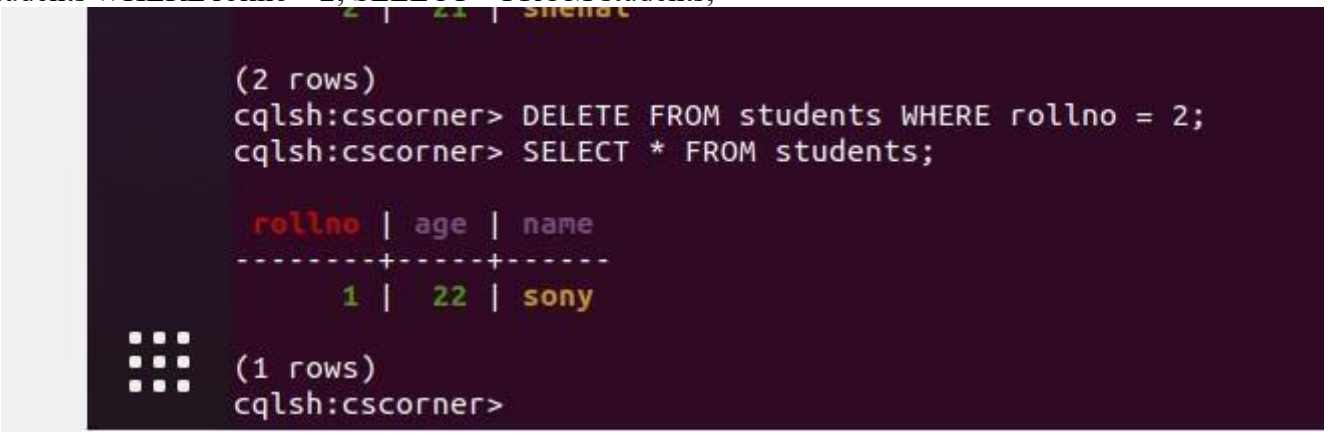
We updated the age of the student with `rollno = 1`. Commands UPDATE

students SET age = 22 WHERE rollno = 1; SELECT * FROM students;



Step 5: Delete Data (Delete)

We deleted the student with `rollno = 2`. DELETE FROM
students WHERE rollno = 2; SELECT * FROM students;



Step 6: Dropping Table and Keyspace

Finally, we cleaned up the environment by dropping the table and the keyspace. DROP TABLE studen

